

ARM PrimeCell

DC-DC Converter Interface (PL160)

Design Manual

ARM PrimeCell DC-DC Converter Interface (PL160) Design Manual

© Copyright ARM Limited 1999. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell DC-DC Converter Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	DC-DC CONVERTER INTERFACE DESIGN OVERVIEW	3
4.1	Functional Description	3
4.2	Signal Description	5
4.3	Programmer's Model	7
4.3.1	Functional Mode Registers	7
4.3.2	Register Memory Map	9
4.4	Block Partitioning	10
4.4.1	APB Interface (DcdcApbif)	10
4.4.2	Control Logic Block (DcdcCntl)	13
4.4.3	Drive Output Logic Block (DcdcDrive)	15
4.4.4	DCDCCLK Synchronisation Block (DcdcClkSync)	16
4.4.5	PCLK Synchronisation Block (DcdcPclkSync)	17
5	DC-DC CONVERTER INTERFACE IMPLEMENTATION DETAILS	19
5.1	Design Hierarchy	19
5.2	DC-DC Converter Interface Top Level Block (Dcdc)	20
5.3	DC-DC Converter Interface APB Interface (DcdcApbif)	20
5.3.1	Write Interface	20
5.3.2	Read Interface	20
5.4	Control Logic Block (DcdcCntl)	21
5.5	Drive Output Logic (DcdcDrive)	22
5.6	PCLK Synchronisation Cells (DcdcPclkSync)	23
5.7	DCDCCLK Synchronisation Cells (DcdcClkSync)	23
6	DC-DC CONVERTER INTERFACE TEST DETAILS	24

6.1	DC-DC Converter Interface Testbenches	24
6.1.1	VHDL Bus Compliance testbenches	24
6.1.1.1	Unit Under Test	28
6.1.1.2	APB Bridge Model	28
6.1.1.3	Protocol Checker	28
6.1.1.4	Vector Sets	28
6.1.1.5	Virtual Registers	29
6.1.1.6	Generics	29
6.1.1.7	Running Simulations	30
6.1.2	Verilog Bus Compliance testbenches	31
6.1.2.1	Unit Under Test	35
6.1.2.2	APB Bridge Model	35
6.1.2.3	Protocol Checker	35
6.1.2.4	Vector Sets	35
6.1.2.5	Virtual Registers	36
6.1.2.6	Parameters	36
6.1.2.7	Running Simulations	37
6.1.3	Unification of BusTalk and TicTalk	38
6.1.4	Vector Generation	39
6.1.5	AMBA TICTalk testbenches	40
6.2	Free Running Functional Verification Tests	42
6.2.1	Register Reset Tests	42
6.2.2	Positive Polarity Drive 0 1800 KHz Functional Test with DRV0Sel = 1	42
6.2.3	Positive Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 0	42
6.2.4	Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1	42
6.2.5	Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 0	43
6.2.6	Negative Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 1	43
6.2.7	Negative Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1	43
6.2.8	FEEDBACK0 CHECK	43
6.2.9	FEEDBACK1 CHECK	43
6.2.10	Drive 0 900 KHz Functional Test	43
6.2.11	Drive 1 900 KHz Functional Test	43
6.2.12	Drive 0 225 KHz Functional Test	43
6.2.13	Drive 1 225 KHz Functional Test	44
6.2.14	Drive 0 95 KHz Functional Test	44
6.2.15	Drive 1 95 KHz Functional Test	44
6.3	Clock-enabled Production Tests	45
6.3.1	Register Reset Tests	45
6.3.2	Counter Operation Test	45
6.3.3	Positive Polarity Drive 0 1800 KHz Functional Test with DRV0Sel = 1	46
6.3.4	Positive Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 0	46
6.3.5	Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1	46
6.3.6	Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 0	46
6.3.7	Negative Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 1	46
6.3.8	Negative Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1	46
6.3.9	FEEDBACK0 Check	46
6.3.10	FEEDBACK1 Check	46
6.3.11	Drive 0 900 KHz Functional Test	46
6.3.12	Drive 1 900 KHz Functional Test	47
6.3.13	Drive 0 225 KHz Functional Test	47
6.3.14	Drive 1 225 KHz Functional Test	47
6.3.15	Drive 0 95 KHz Functional Test	47
6.3.16	Drive 1 95 KHz Functional Test	47

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	14 December, 1998	First Release
B	4 June, 1999	Second release, Verilog testbenches added.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM DDI 0147	ARM PrimeCell DC-DC Converter Interface (PL160) Technical Reference Manual
2	PL160 INTM 0000	ARM PrimeCell DC-DC Converter Interface (PL160) Integration Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
DC-DC	Direct Current-Direct Current (power conversion)
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell DCDC (DC-DC Converter Interface). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the DCDC design, describing the functional partitioning of the DCDC and the signal interconnections. Section 5 presents detailed descriptions on the DCDC design with simulation waveforms. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the DCDC.

3 INTRODUCTION

The DCDC is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell DC-DC Converter Interface (PL160) Technical Reference Manual [Ref.1]. The ARM PrimeCell DC-DC Converter Interface (PL160) Integration Manual [Ref.2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

4 DC-DC CONVERTER INTERFACE DESIGN OVERVIEW

4.1 Functional Description

The ARM PrimeCell DC-DC Converter Interface is an APB slave block and connects to the Advanced Peripheral Bus (APB). This block can be used to implement a configurable dual output pulse-width modulation (PWM) DC-DC power converter. Control/status information is transferred between the micro-controller and the DC-DC Converter Interface block via the AMBA APB to configure the frequency and duty cycle of two outputs DCDCDRIVE0OUT and DCDCDRIVE1OUT.

The two drive outputs, (DCDCDRIVE0OUT and DCDCDRIVE1OUT) are each configurable to switch at one of four fixed frequencies $f_{in}/16$, $f_{in}/32$, $f_{in}/128$ or $f_{in}/304$ where f_{in} is the frequency of the input clock DCDCCLK. For example, an input clock DCDCCLK, having a 28.8MHz frequency will generate output frequencies of 1.8MHz, 900KHz, 225KHz, and 94.7KHz. Each drive output can also be configured with a duty cycle ranging from 0 to $^{15}/_{16}$ in intervals of $^{1}/_{16}$, by writing to the appropriate Configuration Register.

A DC-DC converter can be implemented by connecting a drive output to switch an external power semiconductor in an appropriate analog DC-DC power conversion circuit.

Each drive output is enabled by an external input pins (DCDCFB0 or DCDCFB1) which would normally be connected to the output of an analog comparator monitoring the DC-DC converter output voltage level.

Polarity status signals DCDCDRIVE0IN and DCDCDRIVE1IN allow either positive or negative voltages to be generated by the DC-DC converter. The sense of the DC-DC converter drive output signal (active HIGH or active LOW) is determined by latching the state of the drive output signals during power on reset. A pull-up on the drive signal will result in an active LOW drive output whereas a pull-down resistor will result in an active-HIGH drive output).

Figure 4.1-1 illustrates the functionality of the DC-DC Converter Interface block.

Figure 4.1-1 illustrates the functionality of the DC-DC Converter Interface block.

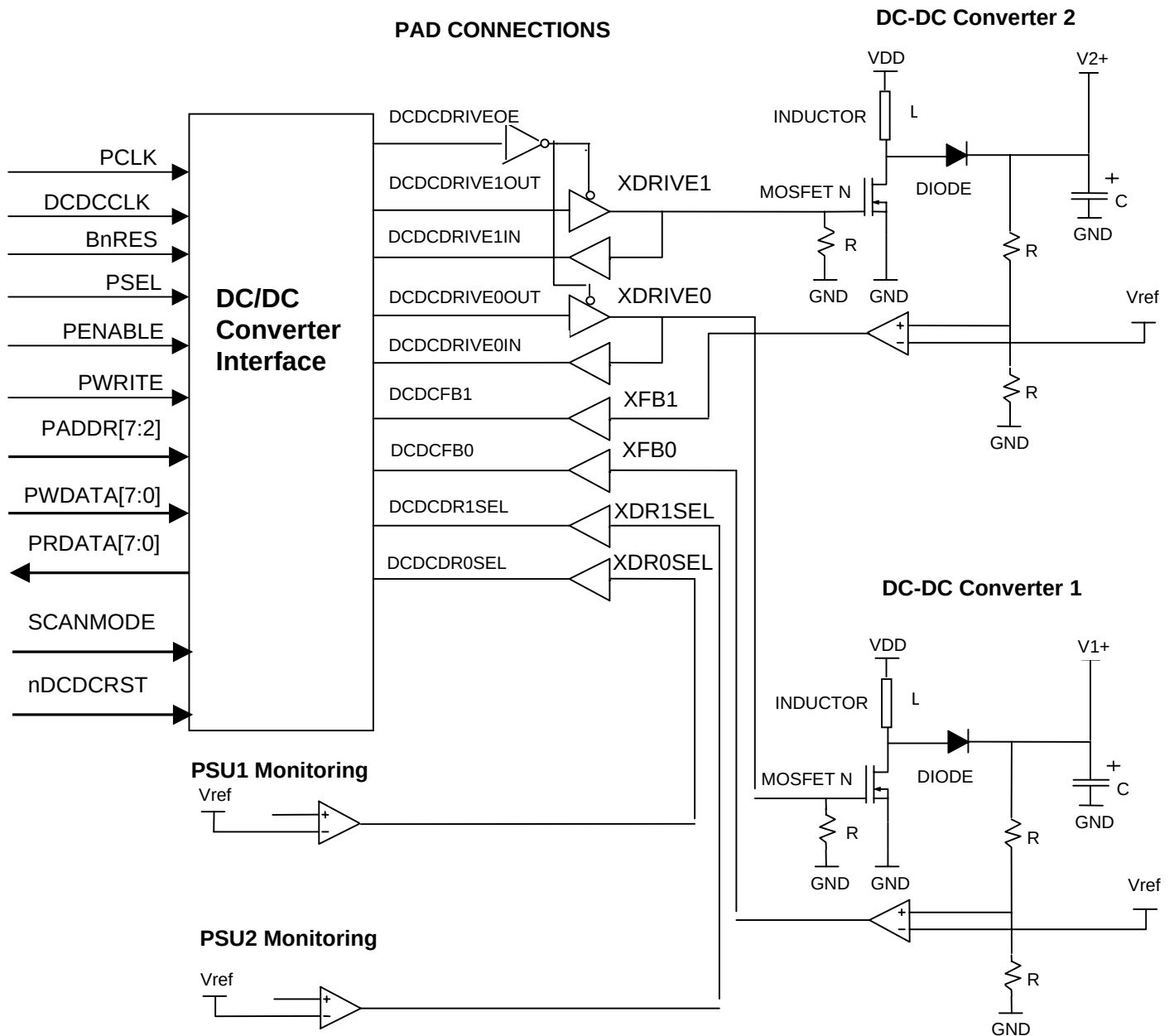


Figure 4.1-1 DC-DC Converter Interface Block Diagram

4.2 Signal Description

The following table describes the DC-DC Converter Interface signals. **Table 4.2-1** lists the APB interface signals **Table 4.2-2** lists the block-specific on-chip non-APB signals and **Table 4.2-3** lists the signals to I/O pad cells.

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Clock input, used to time all bus transfers
BnRES	IN	Reset Controller	Bus reset input signal (active low).
PADDR[7:2]	IN	APB Bridge	Subset of the APB address bus.
PWDATA[7:0]	IN	APB Bridge	Data is written to the block on this bus.
PRDATA[7:0]	OUT	APB Bridge.	Data is read from the block on this bus.
PENABLE	IN	APB Bridge	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the block and when LOW, a read from the block.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the block has been selected by the APB Bridge.

Table 4.2-1: AMBA APB signals

Signal Name	Type	Source / Destination	Description
DCDCCLK	IN	External Clock	DC-DC CONVERTER Reference clock.
nDCDCRST	IN	Reset Controller	Asynchronous reset input signal (active LOW) for the DCDCCLK domain.
SCANMODE	IN	Test Controller	Scan test mode input. This signal must be asserted HIGH during scan testing to ensure that the internal data storage elements can be asynchronously reset. SCANMODE must be negated LOW during normal use or when applying manufacturing test vectors via the TIC.

Table 4.2-2: On-chip signals

Signal Name	Type	Source / Destination	Description
DCDCDRIVE0IN DCDCDRIVE1IN	IN	PAD	Inputs sampled by DC-DC CONVERTER during reset. This sampled value is used to determine DCDCDRIVE0OUT and DCDCDRIVE1OUT output polarity.
DCDCFB0 DCDCFB1	IN	PAD	External feedback inputs from analog power conversion circuit. These inputs enable the drive outputs DCDCDRIVE0OUT and DCDCDRIVE1OUT.
DCDCDR0SEL DCDCDR1SEL	IN	PAD	These signals can be used to switch between two different frequency/duty cycle configurations for each drive output.
DCDCDRIVEOE	OUT	PAD	Output enable for tri-state buffers of DCDCDRIVE0OUT and DCDCDRIVE1OUT outputs.
DCDCDRIVE0OUT DCDCDRIVE1OUT	OUT	PAD	Drive outputs with software configurable frequency and duty cycle to switch an off-chip power semiconductor.

Table 4.2-3: Block specific signals to I/O pads

4.3 Programmer's Model

4.3.1 Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
DcdcBase + 0x00	R/W	8	0x00	PMPCON0	Writes to this register will load the duty cycle data for DCDCDRIVE0OUT. Reads from this register will return data contained within the PMPCON0.
DcdcBase + 0x04	R/W	8	0x00	PMPCON1	Writes to this register will load the duty cycle data for DCDCDRIVE1OUT. Reads from this register will return data contained within the PMPCON1.
DcdcBase + 0x08	R/W	8	0x00	PMPFREQ	Writes to this register will load frequency configuration data for both the DRIVE outputs. i.e. DCDCDRIVE0OUT & DCDCDRIVE1OUT. Reads from this register will return data contained within the PMPFREQ.
DcdcBase + 0x0c-3c					Reserved.
DcdcBase + 0x40-a0					Reserved (for test purposes).
DcdcBase + 0xa4-ff					Reserved.

Table 4.3-1 ARM PrimeCell DCDC Register Summary

Please see the ARM PrimeCell DC-DC Converter Interface (PL160) Technical Reference Manual [Ref. 1] for a detailed description of each register.

Test Mode Registers

Address	Type	Width	Reset Value	Name	Description
DcdcBase + 0x40 -0x7c	R/W	0	-	PMPTCER	Test Clock Enable Register
DcdcBase + 0x80	R/W	5	b--- 00000	PMPTCR	Test Control Register
DcdcBase + 0x84	R/W	2	b----- 00	PMPTMR	Test Mode Register
DcdcBase + 0x88	R/W	6	b-- 000000	PMPTISR	Test Input Stimulus Register
DcdcBase + 0x8c	R	3	b----- 100	PMPTOCR	Test Output Capture Register
DcdcBase + 0x90	R	5	b--- 00000	PMPFC0	Frequency Counter 0 read Register
DcdcBase + 0x94					Reserved
DcdcBase + 0x98	R	5	b--- 00000	PMPFC1	Frequency Counter 1 read Register
DcdcBase + 0x9c					Reserved
DcdcBase + 0xa0	R	8	b00000 000	PMPDRVCNT	Pulse width counter values read Register

Table 4.3-2 ARM PrimeCell DCDC Test Mode Registers

Please see ARM PrimeCell DC-DC Converter Interface (PL160) Technical Reference Manual [Ref.1] for a detailed description of each register.

4.3.2 Register Memory Map

Address	Read Location	Write Location
DCDC Base + 0x00	PMPCON0	PMPCON0
DCDC Base + 0x04	PMPCON1	PMPCON1
DCDC Base + 0x08	PMPFREQ	PMPFREQ
DCDC Base + (0x0c to 0x3c)	Reserved	Reserved
DCDC Base + (0x40 to 0x7c)	PMPTCER (Test Mode Only)	PMPTCER (Test Mode Only)
DCDC Base + 0x80	PMPTCR (Test Mode Only)	PMPTCR (Test Mode Only)
DCDC Base + 0x84	PMPTMR (Test Mode Only)	PMPTMR (Test Mode Only)
DCDC Base + 0x88	PMPTISR (Test Mode Only)	PMPTISR (Test Mode Only)
DCDC Base + 0x8c	PMPTOCR (Test Mode Only)	
DCDC Base + 0x90	PMPFC0 (Test Mode Only)	
DCDC Base + 0x94	Reserved	Reserved
DCDC Base + 0x98	PMPFC1 (Test Mode Only)	
DCDC Base + 0x9c	Reserved	Reserved
DCDC Base + 0xa0	PMPDRVCNT (Test Mode Only)	
DCDC Base + (0xa4 to 0xfc)	Reserved	Reserved

Table 4.3-3 ARM PrimeCell DCDC Register Memory Map

4.4 Block Partitioning

The sub-blocks in the design are listed below.

- APB interface (Input module and Output module)
- Control logic block
- Drive output Logic block
- DCDCCLK synchronisation block
- PCLK synchronisation block

The design of the above-mentioned sub-blocks is detailed below.

4.4.1 APB Interface (DcdcApbif)

The APB interface consists of the input module and the output module. This interface deals with reads and writes to registers from the APB. A simplified block diagram of the APB interface block is shown in **Figure 4.4-1**.

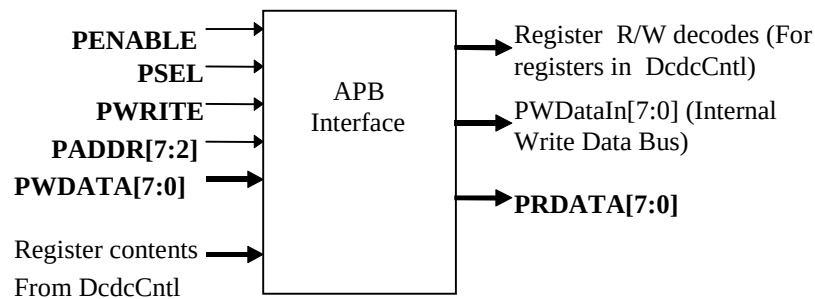


Figure 4.4-1 APB Interface Block diagram

Input Logic

The APB write interface generates decodes for reads and writes to all registers in the device. Each register addressable from the APB, is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode and PENABLE, PSEL and PWRITE signals.

The block diagram in **Figure 4.4-2** illustrates the structure of the input logic within the Control (DcdcCntl) block. In this block diagram, the write decodes are generated in the APB interface block, (DcdcApbif) while the registers are actually located in the DcdcCntl block.

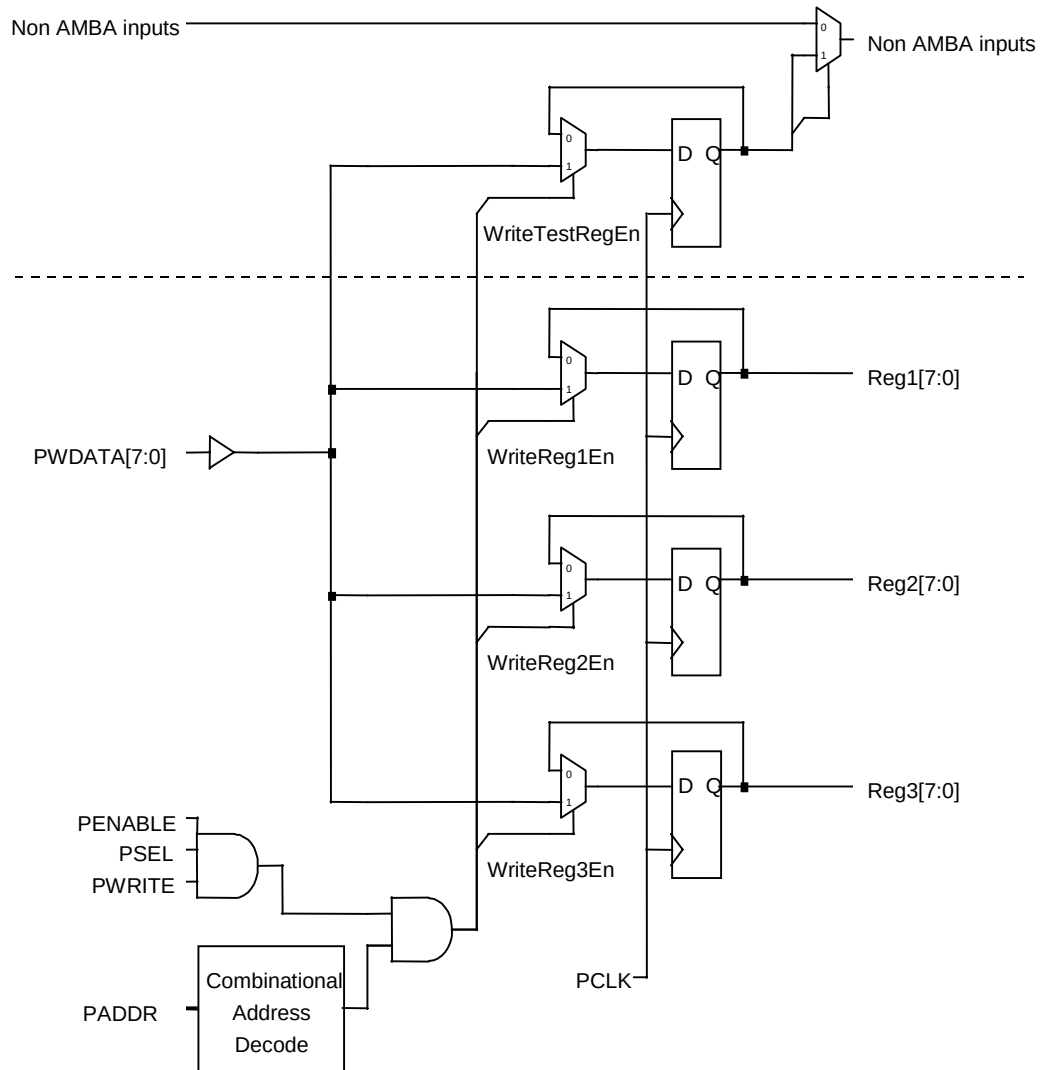


Figure 4.4-2 Input Logic

Output Logic

The output logic in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read data bus. A combinatorial address decode is used to select which of the output sources are driven onto the bus during a read request. Non-AMBA outputs such as DCDCDRIVE0OUT, DCDCDRIVE1OUT and DCDCDRIVEOE are also made observable through APB read accesses. The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of external bus connection method used for the data bus on the APB. A block diagram of the output module is shown in **Figure 4.4-3**. There is a register on the output path from the output mux. This ensures that the outputs are latched on the rising edge of PCLK. The registers shown in this block diagram are actually located in the DcdcCntl block, while the output multiplexer is located in the DcdcApbif Block.

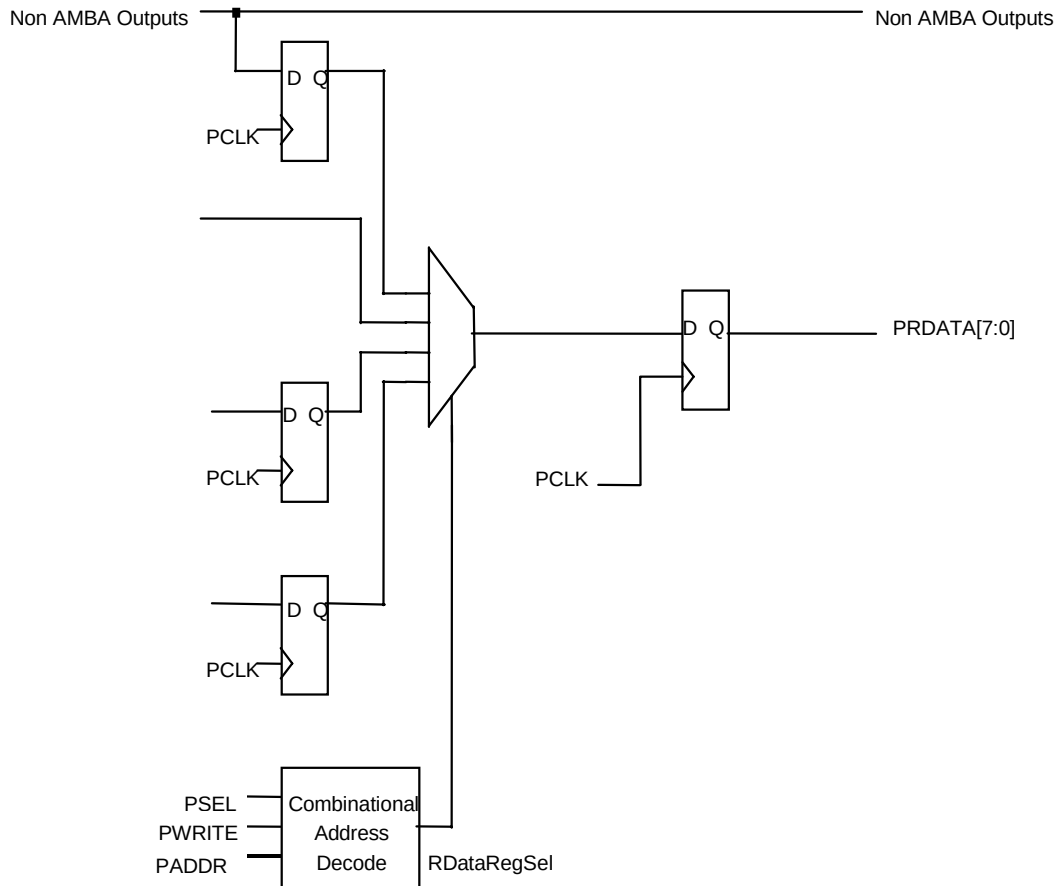


Figure 4.4-3 Output Logic

Test Logic

The DcdcApbif block also contains, in addition to the read logic for all writeable Registers, the 4 more Test Mode Registers, which are a part of the DC-DC Converter Interface Test Logic. These are the PMPFC0, PMPFC1, PMPDRVCNT and PMPTOCR registers.

PMPFC0 Register

The PMPFC0 register contains the value of Frequency Counter 0 located in the DcdcDrive Block. The Frequency Counter counts up to the value programmed in the Frequency Pointers and generates a pulse, one DCDCLK cycle wide.

PMPFC1 Register

The PMPFC1 register contains the value of Frequency Counter 1 located in the DcdcDrive Block. The Frequency Counter counts up to the value programmed in the Frequency Pointers and generates a pulse, one DCDCLK cycle wide.

PMPDRVCNT Register

The PMPDRVCNT register contains the values of the 2 Drive Counters; Drive 0 Counter and Drive 1 Counter located in the DcdcDrive Block. These counters count the Freq16x* pulses. By counting the

Freq16x* pulses up to the selected Duty Cycle number, the appropriate Drive Output duty cycle is generated.

PMPTOCR Register

The PMPTOCR register is used to observe the primary non-AMBA outputs DCDCDRIVEOE, DCDCDRIVE0OUT and DCDCDRIVE1OUT generated by the DcdcDrive Block. In the Test Mode this register is read to ensure the correct functional operation of the DC-DC Converter Interface.

4.4.2 Control Logic Block (DcdcCntl)

Functional Logic

The DcdcCntl Block contains three (8 bit) Registers (PMPCON0, PMPCON1 and PMPFREQ). Each of the PMPCON* registers contain the following for their respective drive output:

- Output Duty Cycle when DCDCDR*SEL is Low (4 bits)
- Output Duty Cycle when DCDCDR*SEL is High (4 bits)

The PMPFREQ register contains the following for the drive output frequencies:

- Drive 0 Frequency when DCDCDR0SEL is low (2 bits)
- Drive 0 Frequency when DCDCDR0SEL is high (2 bits)
- Drive 1 Frequency when DCDCDR1SEL is low (2 bits)
- Drive 1 Frequency when DCDCDR1SEL is high (2 bits)

This allows completely independent control of each drive output.

In addition to the configuration registers, the DcdcCntl Block also contains muxes whose output may be switched between the two possible duty cycles and the two frequencies stored in each configuration register. These muxes are controlled by the respective DCDCDR*SEL lines.

The DcdcCntl Block also contains the logic necessary to produce the 3 Update signals, FREQUpdate, CON0Update and CON1Update. These 3 signals are synchronised to the DCDCCLK in the DcdcClkSync Block and then passed on to the DcdcDrive Block where they are used to produce the write enable signals for the DCDCFREQ, DCDCCON0 and DCDCCON1 registers. In essence, these 3 signals are synchronisation signals synchronised to the DCDCCLK and used in DcdcDrive Block to produce the Drive Outputs.

The DcdcCntl also contains drive polarity registers which latch in the data present on the DCDCDRIVE0IN and DCDCDRIVE1IN registers when DCDCDRIVEOE is low to determine the drive outputs polarity. The outputs of these registers are used as control lines to muxes that will switch drive output polarities between active high and active. The DCDCDRIVEOE signal is generated by registering nRES (System Reset). Since the DCDCDRIVEOE signal is derived from the nRES signal it is not a resetable register. Also, since the drive polarity registers must latch in data during the reset sequence, they are not reset by the input signal.

System Reset

The nDCDCRES (System Reset) is generated by logically ORing BnRES input signal with an inverted TestRst (Test Reset), bit 3 of the PMPTCR (Test Control Register). This allows all functional registers to be reset either by hardware or software.

Test Logic

The DcdcCntl Block contains eight registers used for testing the DC-DC Converter Interface. These registers are the Test Clock Enable Register, the Test Control Register (PMPTCR), the Test Mode Register (PMPTMR), the Test Input Stimulus Register (PMPTISR), the Test Output Capture Register (PMPTOCR), the Frequency Counter0 Read Register (PMPFC0), the Frequency Counter1 Read Register (PMPFC1), and the Pulse Width Counter Read Register (PMPDRCNT).

Test Clock Enable Register

This is a virtual register that is selected for address offsets from 0x40 to 0x7C. In the Registered clockmode, the clock enable (ClkEn) is pulsed for one clock cycle each time this register is accessed. Reads return zeros and write data is ignored.

Test Control Register

This register specifies the clocking scheme to be used in the test mode. In test mode, all clock inputs to the peripheral are driven by the same signal. The test mode is said to have been entered when the TestModeEn bit is set to 1. In the test mode, if the TestClkEn bit is set to 1, Test clocking is said to be enabled. When Test clocking is enabled, if the RegClkMode bit is 0, a pulse is generated on the ClkEn signal on every access to the peripheral. When Test clocking is enabled, if the RegClkMode bit is 1, a pulse is generated on the ClkEn signal only when the Test Clock Enable Register (PMPTCER) is accessed.

The TestRst bit in the register allows the flip-flops in the design to be asynchronously reset in the Test mode. This helps in avoiding X propagation in simulations and to reset the internal registers during production level testing.

The TestInpSel bit selects the source for the internal input signal for external non-AMBA inputs. When this bit is set to 0, the external inputs from the DCDCDRIVE0IN, DCDCDRIVE1IN, DCDCFB0, DCDCFB1 DCDCDR0SEL and DCDCDR1SEL lines are chosen to drive the internal lines. When the TestInpSel bit is set to 1, The values programmed in the Test Input Stimulus Register are used to drive the internal lines.

Test Mode Register

This register is used to put the counters into 'nibble mode' to reduce the number of clock cycles required for test.

Test Input Stimulus Register

This register provides direct stimulus control of the non-AMBA primary inputs. Each bit in this register corresponds to an equivalent primary input. Depending upon the test mode the device is in, either the data in this register or the data on the primary input is applied as the stimulus for the device. When a read is performed from this register, the data read back is dependent on the status of the TestInpSel bit of the Test Control Register. When the TestInpSel bit is set to 0, the external inputs from the DCDCDRIVE0IN, DCDCDRIVE1IN, DCDCFB0, DCDCFB1 DCDCDR0SEL and DCDCDR1SEL lines are read. When the TestInpSel bit is set to 1, The values programmed in the Test Input Stimulus Register are read.

Test Output Capture Register

Reads from this register provide observation of the peripheral non-AMBA primary outputs. This includes the DCDCDRIVE0OUT, DCDCDRIVE1IOUT and DCDCDRIVEOE.

Frequency Counter 0 Read Register

This is a read only register which enables the status of Frequency Counter 0 to be read.

Frequency Counter 1 Read Register

This is a read only register which enables the status of Frequency Counter 1 to be read.

Pulse Width Generator Counter Read Register

This is a read only register which enables the status of Pulse Width Generator Counter 0 to be read.

System Clock Enable

Logic to generate the system clock enable (ClkEn) is also located in the DcdcCntl block. For a description of this circuitry refer to the Test Control Register Description.

4.4.3 Drive Output Logic Block (DcdcDrive)

The Drive Output Logic Block (DcdcDrive) converts the frequency and duty cycles coefficients provided by the DcdcCntl block into pulse-width modulated outputs (DCDCDRIVE0OUT and DCDCDRIVE1OUT). This is done by 2 counters for each Drive Output. The first counter brings in the DCDCCLK and divides it to create an output pulse that has a frequency 16 times that of the final drive output frequency. This output signal is then sent to a second counter where it is counted to create the proper output frequency and duty cycle. This block also handles the combinatorial muxing for the test input select bit of the PMPTCR register. In addition, this block also has the FREQUpdateSync, CON0UpdateSync and CON1UpdateSync signals as inputs which are used to update the internal registers holding the Duty Cycle Configuration and Frequency information.

In addition, the DcdcDrive Block also contains muxes whose output may be switched between the two possible duty cycles and the two frequencies stored in each configuration register. These muxes are controlled by the respective DCDCDR*SEL lines. Logic is also provided to switch the drive outputs HIGH and LOW based on the value of the (DCDCFB0 and DCDCFB1) inputs. The DcdcCntl also contains drive polarity registers which latch in the data present on the DCDCDRIVE0IN and DCDCDRIVE1IN registers when DCDCDRIVEOE is low to determine the drive outputs polarity. The outputs of these registers are used as control lines to muxes that will switch drive output polarities between active high and active low. The DCDCDRIVEOE signal is generated by registering nDCDCRES (DCDC domain reset). Since the DCDCDRIVEOE signal is derived from the nDCDCRES signal it is not a resettable register. Also, since the drive polarity registers must latch in data during the reset sequence, they are not resettable either.

Frequency Logic

The Frequency Logic is made up of the frequency coefficient decoders and the frequency counters. The coefficient decoders read in the frequency coefficient for each drive output supplied by the DcdcCntl and convert these coefficients into frequency pointers which represent the number of DCDCCLK cycles it takes to make up $1/16^{\text{th}}$ of the final drive outputs period. These frequency pointers are then passed on to the frequency counter. The frequency counter is a free running counter which counts up to the assigned frequency pointer and then generates a pulse (Freq16x) one DCDCCLK cycle wide. This pulse is passed on to the Duty Cycle Logic.

For instance, if the DCDCCLK input frequency is not 28.8MHz or the drive output frequencies required are other than 1.8MHz, 900KHz, 225KHz or 94.7KHz, then the frequency pointers found in the frequency decoder must be modified to accommodate the new output frequencies. Depending on the size of the frequency pointer values, register sizes may need to be altered as well.

Duty Cycle Logic

The duty cycle counters are free running 4-bit counters, which are reset by the DCDC domain nDCDCRES signal. If the duty cycle counter value is less than the duty cycle coefficient supplied by the DcdcCntl block the drive output is set to one, otherwise it is set to zero. The duty cycle counter value is incremented by one every time it is enabled by a Freq16x pulse from the Frequency Logic. This results in an output of the defined frequency and duty cycle.

Figure 4.4-6 shows the block diagram of the Drive Output Logic.

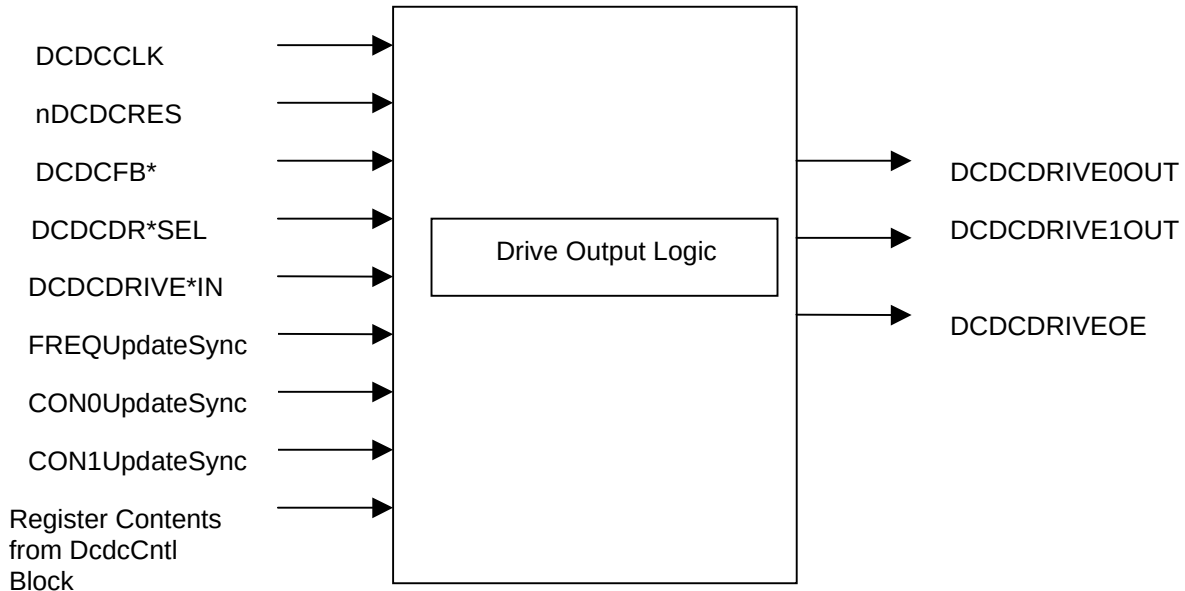


Figure 4.4-6 Drive Output Logic

4.4.4 DCDCCLK Synchronisation Block (DcdcClkSync)

Functional Logic

Frequency and Duty Cycle update signals are passed into the DcdcClkSync block from the DcdcCntl block. These signals are immediately passed through Synchronisation Cells to synchronise the incoming coefficients to the DCDCCLK domain and passed on through to the DcdcDrive block. These synchronisation cells are comprised of two D-type registers connected in series and both clocked by DCDCCLK. Asynchronous data flows into the D inputs of the first register and synchronous data flows from the Q outputs of the second register. Each flip-flop in all synchronisation cells may be reset by the System Reset (nDCDCRES)

Figure 4.4-7 shows the block diagram of the Synchronisation Cells.

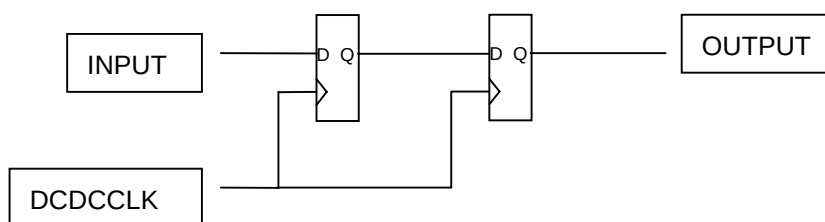


Figure 4.4-7 Synchronisation Cells

Figure 4.4-8 shows the block diagram of the DcdcClkSync Block.

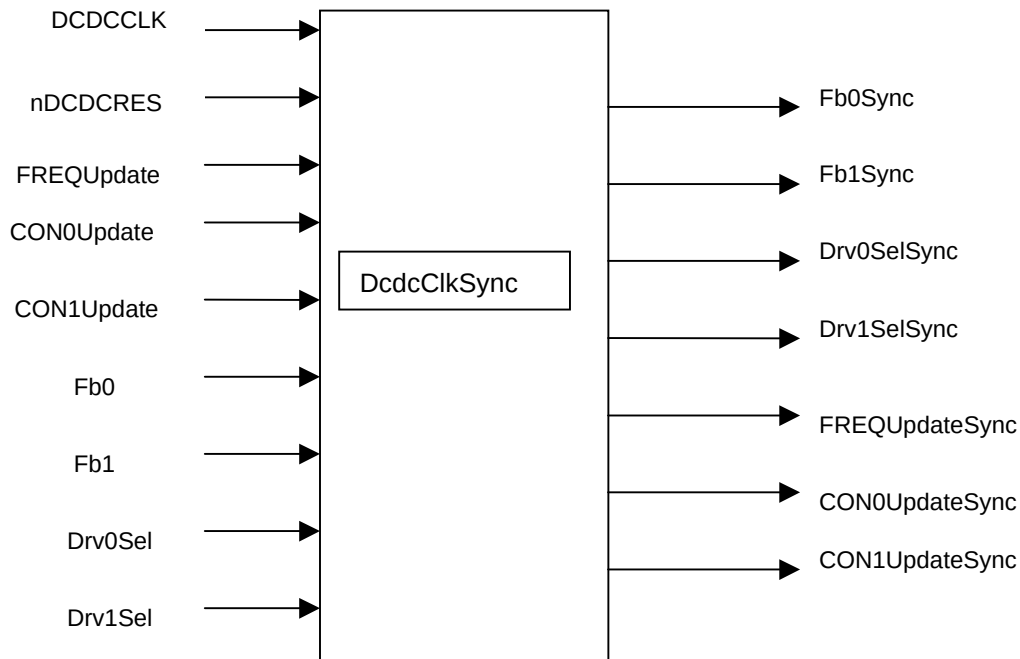


Figure 4.4-8 DcdcClkSync Block.

4.4.5 PCLK Synchronisation Block (DcdcPclkSync)

Functional Logic

Several synchronisation cells are instantiated within the DcdcPclkSync block. These cells take signals that are asynchronous to the PCLK domain and synchronise them to PCLK. These synchronisation cells are comprised of two D-type registers connected in series and both clocked by PCLK. Asynchronous data flows into the D inputs of the first register and synchronous data flows from the Q outputs of the second register. Each flip-flop in all synchronisation cells may be reset by the System Reset (nRES). This block uses the instantiations of the same Synchronisation Cells that are used by DCDCLKSync block for synchronising the signals.

Figure 4.4-9 shows the block diagram of the Synchronisation Cells

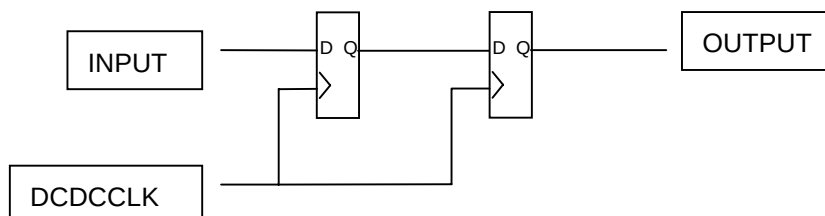


Figure 4.4-9 Synchronisation Cells

Figure 4.4-10 shows the block diagram of the DcdcPclkSync Block.

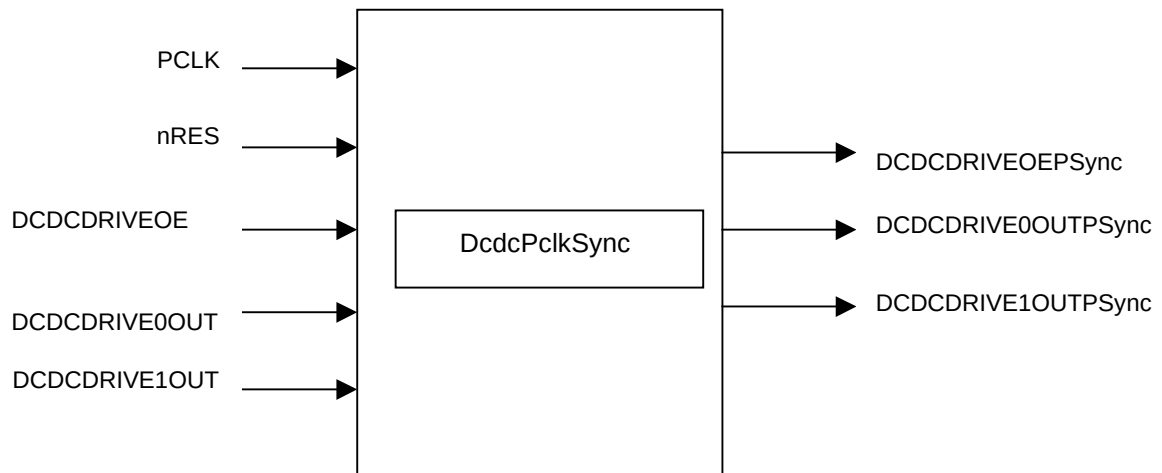


Figure 4.4-10 DcdcPclkSync Block.

5 DC-DC CONVERTER INTERFACE IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The hierarchy in the DC-DC Converter Interface is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-blocks is also given within brackets.

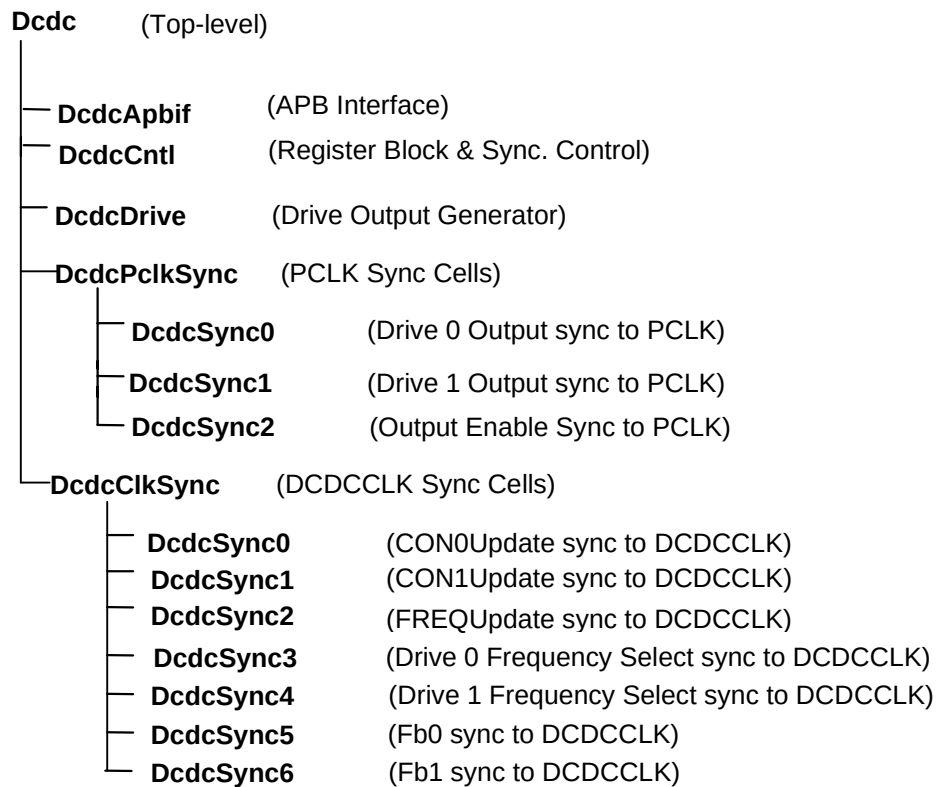


Figure 5.1-1 DC-DC Converter Interface Design Hierarchy

5.2 DC-DC Converter Interface Top Level Block (Dcdc)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the DC-DC Converter Interface.

5.3 DC-DC Converter Interface APB Interface (DcdcApbif)

The APB Interface generates decodes for writes into the registers in the address space of the DC-DC Converter Interface. It also contains the read interface for the DC-DC Converter Interface.

An internal write data bus, PWDATAIn[7:0], is generated by gating the write data bus input to the DC-DC Converter Interface, PWDATA[7:0], with PSEL and PWRITE. This prevents the data bus internal to the DC-DC Converter Interface from toggling when the device is not being written into or when it is not selected.

5.3.1 Write Interface

The write strobe for each register is generated with a decode of PSEL, PWRITE, PADDR[7:0] and PENABLE.

Figure 5.3-1 illustrates the generation of a write strobe for writes into the PMPCON0 register and the latching of data into the register. Data is clocked in to PMPCON0 on the rising edge of PCLK on which the Write enable (PMPCON0WrEn) is asserted. The write enable is derived from a decode of PADDR[7:0], PWRITE, PSEL and PENABLE.

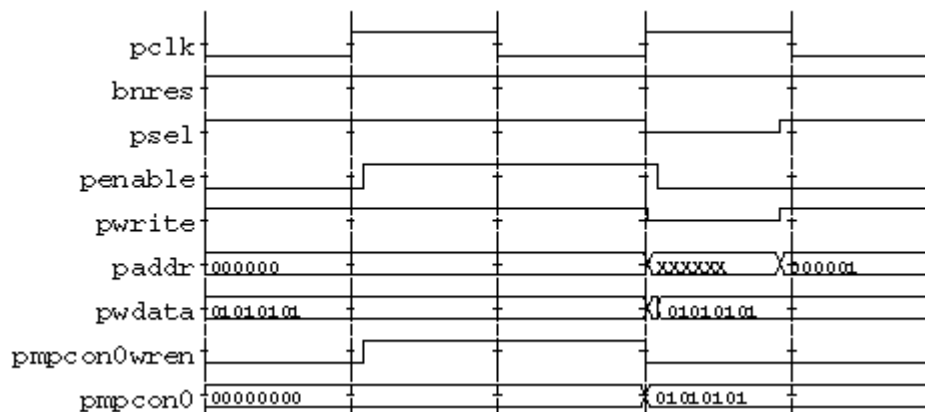


Figure 5.3-1 APB Write Access example

5.3.2 Read Interface

The read interface involves an output multiplexer which drives data onto the output register which in turn drives the data onto the bus at the rising edge of PCLK. Select inputs to the output multiplexer are generated with a combinatorial decode of PWRITE, PA and PSEL signals. Since these signals are stable from the rising edge of PCLK, almost one full PCLK period is available for the output of the multiplexer to stabilise before data is clocked in on the next rising edge of PCLK. Data is sampled by the APB Bridge on the next rising edge of PCLK on which PENABLE is de-asserted.

The external data bus from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexer bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus

when it is not selected. In the DC-DC Converter Interface, the output multiplexer is left such that, when the device is not selected, it drives zeroes onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

Figure 5.3-2 illustrates a read from the PMPCON0 register.

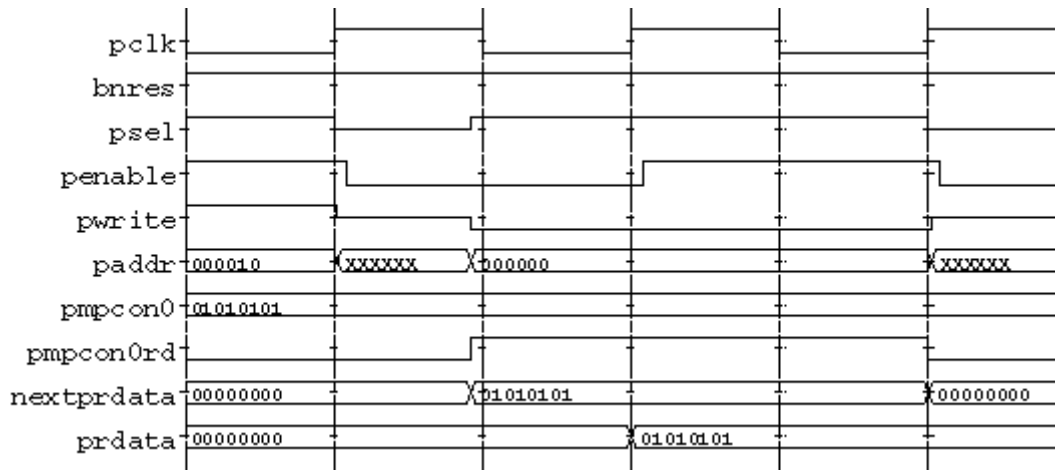


Figure 5.3-2 APB Read Access example

5.4 Control Logic Block (DcdcCntl)

This module contains the Control and Test Registers for the DC-DC Converter Interface. It also contains the combinatorial logic to generate the ClkEn signal (System Clock Enable) and the logic to generate the reset.

The DC-DC Converter Interface makes use of a special handshake synchronisation methodology. This synchronisation methodology is used for multi-bit busses to reduce synchronisation circuitry gate count and eliminate meta-stability. This methodology is implemented for the configuration and frequency registers. As seen in the figure below, DcdcCntl Block produces the FREQUpdate signal in the PCLK domain that is then synchronised to the DCDCLK domain in the DcdcClkSync Block. This synchronised signal (FREQUpdateSync) is then used in the DcdcDrive Block to produce FREQStg2WrEn signal. The FREQStg2WrEn signal is then used to update the DCDCFREQ register that holds the frequency information for the Drive Outputs.

Figure 5.4-1 illustrates the an example of the synchronisation method.

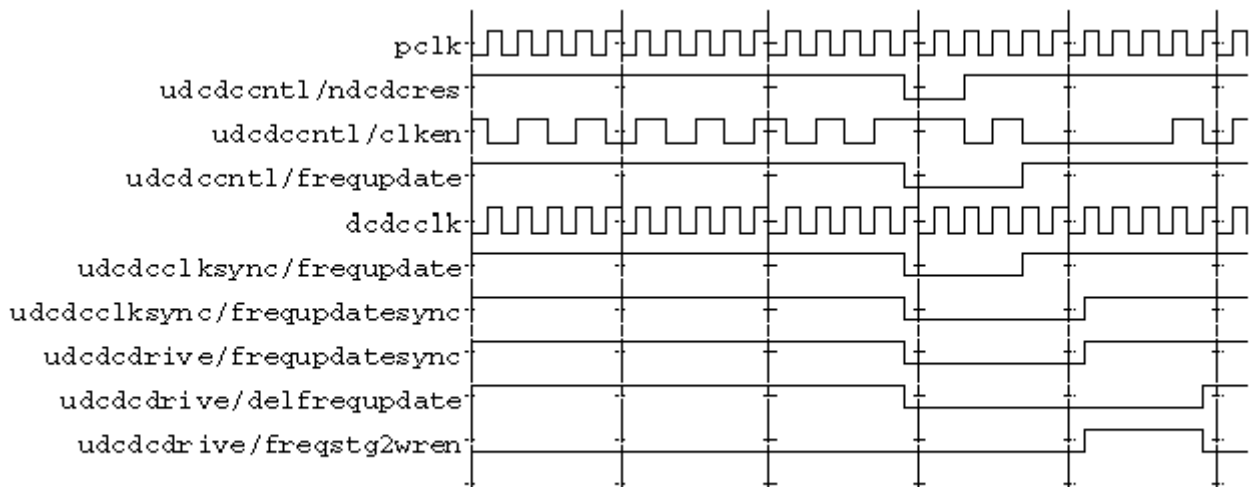


Figure 5.4-1 Example of the Synchronisation method

5.5 Drive Output Logic (DcdcDrive)

This block of the DC-DC Converter Interface is responsible for generating the appropriate waveform for each Drive output depending upon the Duty Cycles & Frequency that has been programmed in the Configuration Registers. For instance if the duty cycle were programmed to a 1 and the frequency to a divide by 32, the illustration below provides insight to the internal signals. As shown in the figure, the Frequency pointer (FreqPtr0) has been set to 1, the Frequency Counter (FreqCnt0) counts up to the value of the FreqPtr0. When FreqCnt0 is equal to FreqPtr0 a pulse is generated on the Freq16x0 signal. Each pulse on the Freq16x0 signal increments the Drive 0 Counter (Drv0Cnt). When Drv0Cnt equals the programmed Duty Cycle, a pulse is generated on the Drv0 signal. If the Drive0 Polarity signal (Drv0Pol) is high DCDCDRIVE0OUT signal is an inverted Drv0 signal. If Drive0 Polarity signal (Drv0Pol) is low DCDCDRIVE0OUT signal is the Drv0 signal. While not visible in the example, the FreqCnt0 is counting up to the FreqPtr0 and hence producing the Freq16x0 pulse.

Figure 5.5-1 illustrates an example of the drive output waveforms.

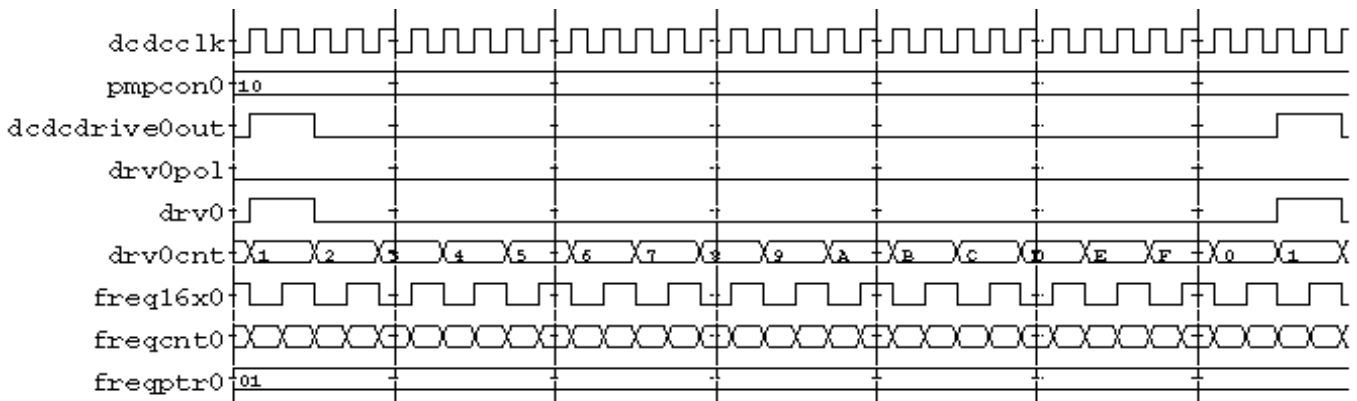


Figure 5.5-1 DCDCDRIVE0OUT Example

5.6 PCLK Synchronisation Cells (DcdcPclkSync)

This block handles the synchronisation of certain external inputs to the PCLK domain. For this purpose this instantiates the following Synchronisation Blocks. All the Synchronisation Blocks are instantiations of the DcdcSync Block.

- DcdcSync0. This block synchronises the DCDCDRIVE0OUT signal to the PCLK domain.
- DcdcSync1. This block synchronises the DCDCDRIVE1OUT signal to the PCLK domain.
- DcdcSync2. This block synchronises the FEEDBACK0 signal to the PCLK domain.
- DcdcSync3. This block synchronises the FEEDBACK1 signal to the PCLK domain.
- DcdcSync4. This block synchronises the DRV0SEL signal to the PCLK domain.
- DcdcSync5. This block synchronises the DRV1SEL signal to the PCLK domain.

5.7 DCDCCLK Synchronisation Cells (DcdcClkSync)

This block synchronises the Duty Cycle & Frequency signals to the DCDCCLK domain. For the purpose of Synchronisation this block contains instantiations of the following blocks.

- DcdcSync0. This block synchronises the CON0Update signal to DCDCCLK.
- DcdcSync1. This block synchronises the CON1Update signal to DCDCCLK.
- DcdcSync2. This block synchronises the FREQUUpdate signal to DCDCCLK.
- DcdcSync3. This block synchronises the Drive 0 Frequency Select to DCDCCLK
- DcdcSync4. This block synchronises the Drive 1 Frequency Select to DCDCCLK
- DcdcSync5. This block synchronises the Fb0 signal to DCDCCLK
- DcdcSync6. This block synchronises the Fb0 signal to DCDCCLK

6 DC-DC CONVERTER INTERFACE TEST DETAILS

6.1 DC-DC Converter Interface Testbenches

6.1.1 VHDL Bus Compliance testbenches

The ARM PrimeCell DC-DC Converter Interface VHDL test world uses two variants of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbenches are designed to be used with most common VHDL simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbenches are “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Production Testbench

Figure 6.1-1 illustrates the VHDL testbench for running production test vectors.

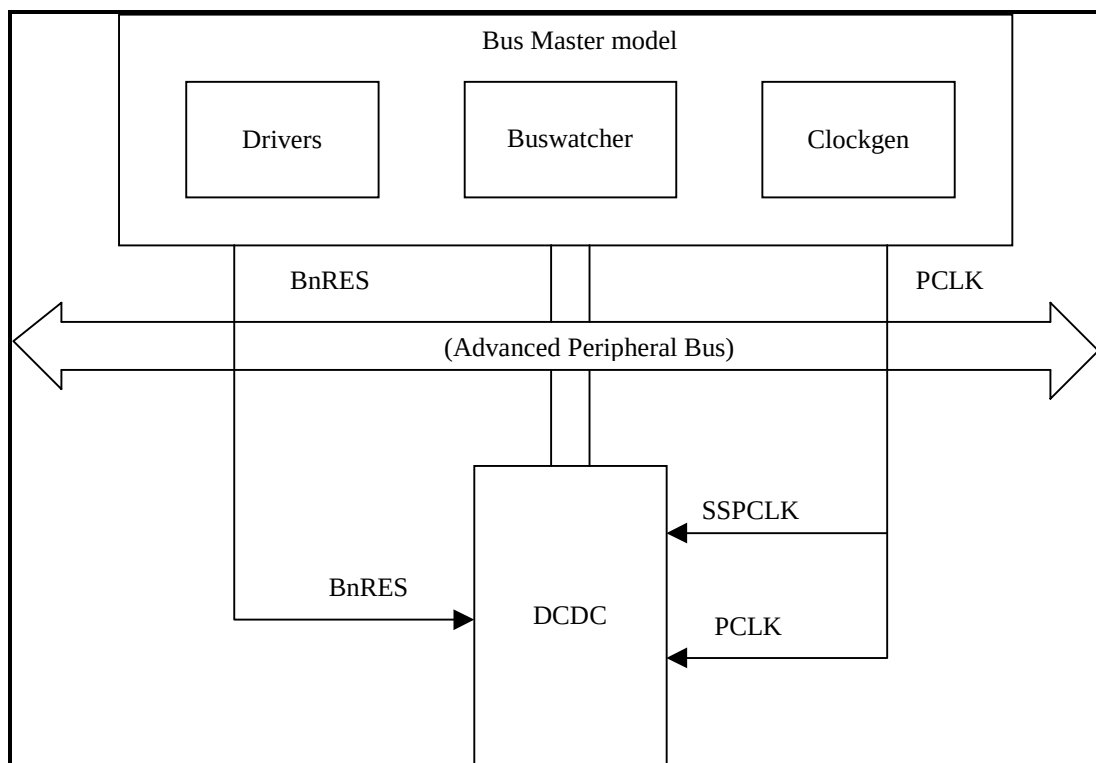


Figure 6.1-1 VHDL testbench for running production test vectors

Figure 6.1-2 illustrates the design hierarchy for the production testbench.

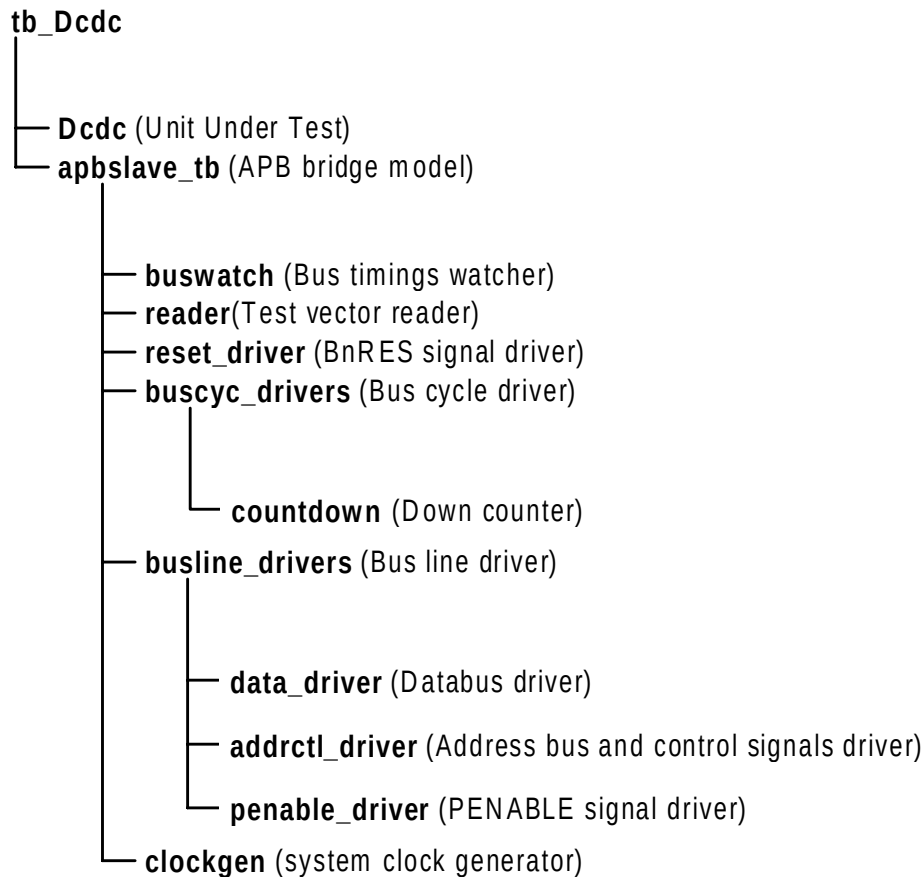


Figure 6.1-2 Testbench hierarchy for production vectors

The testbench, **tb_Dcdc.vhd** may be used for production level testing using internal clock enabling mechanisms. This block instantiates the DC-DC Converter Interface (Unit Under Test), Virtual Registers block and the behavioural model for the APB bridge. The key point to remember about this testbench is that both PCLK and DCDCCLK are fed from the system Bus clock. Since production test vectors use clock enabling to control internal device activity, these two clocks need to run at the same frequency to keep test code frequency independent.

The SCANMODE pin is tied low to keep it from interfering with the test process.

The default test frequency is set at 50MHz. This is exactly the same frequency to which the DCDC was constrained during synthesis. PCLK and DCDCCLK are set to the BCLK frequency defined in **tb_Dcdc.vhd** using the **tcclk** and **tcclk** generic values when instantiating **apbslave_tb.vhd**.

Functional Verification Testbench

Figure 6.1-3 illustrates the VHDL testbench for tests using free-running clocks.

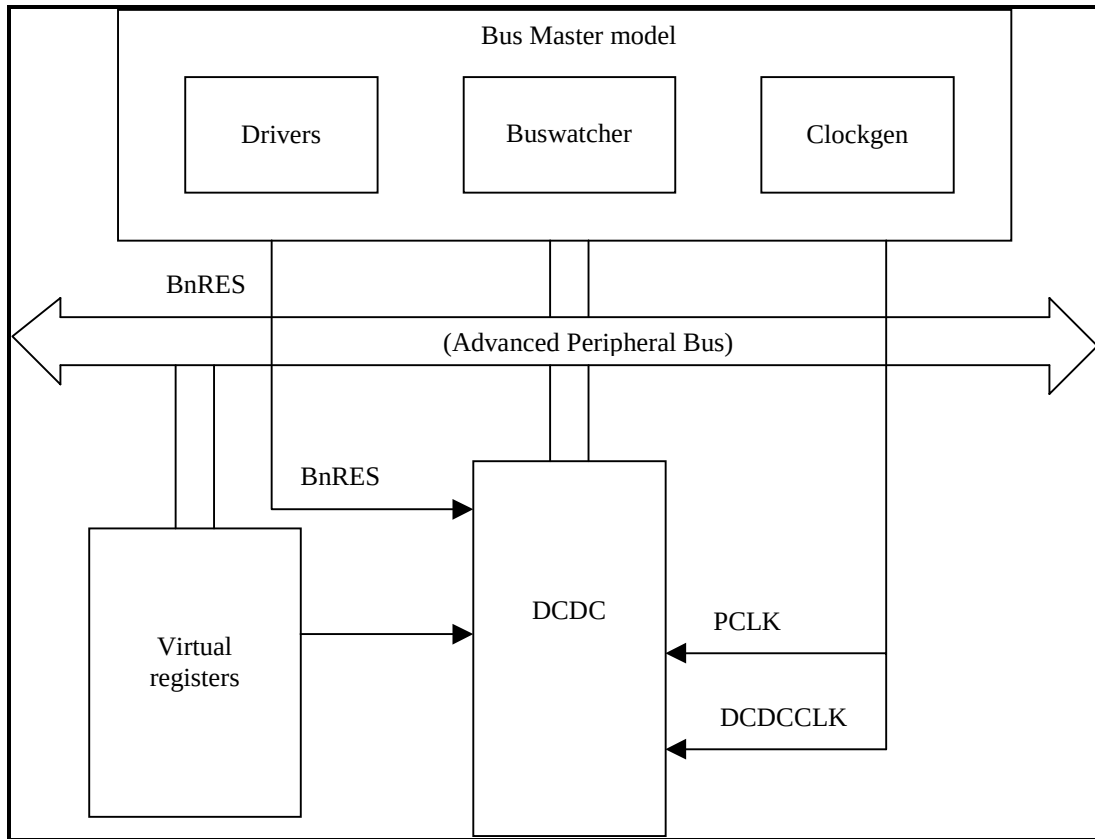


Figure 6.1-2 VHDL testbench for running test vectors using free-running clocks

Figure 6.1-4 illustrates the hierarchy for free-running test vectors.

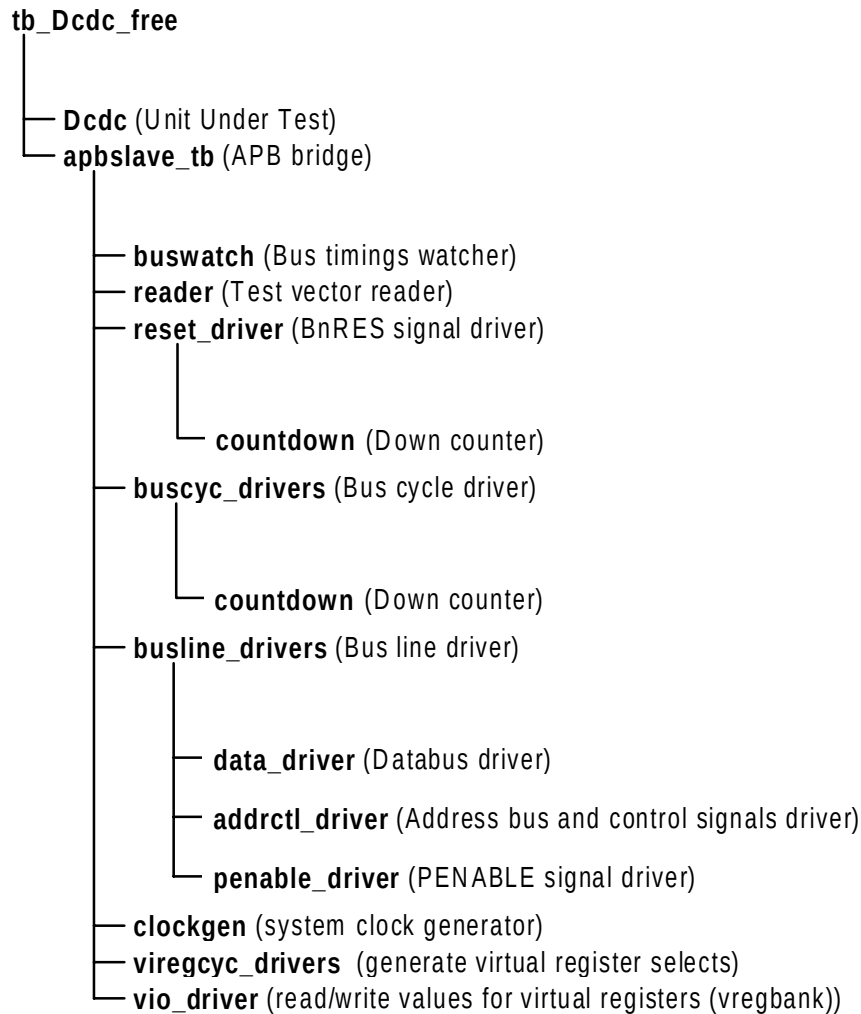


Figure 6.1-4 Testbench hierarchy for free-running vectors

The testbench, **tb_Dcdc_free.vhd** may be used for functional simulations of the DCDC. This module instantiates the DC-DC Converter Interface (Unit Under Test), Virtual Registers block and the behavioural model for the APB bridge. Unlike the case of **tb_Dcdc.vhd**, PCLK and DCDCCLK need not be the same frequency.

All non-AMBA inputs to the DCDC block are driven by the virtual registers.

The SCANMODE pin is tied low to keep it from interfering with the test process.

The PCLK frequency has been set to 50 MHz (This is exactly the same frequency to which the DCDC was constrained during synthesis) and DCDCCLK was set to 25 MHz. The free-running test program requires a fixed 2:1 clock frequency ratio for PCLK to DCDCCLK.

PCLK is set to the BCLK frequency defined in **tb_Dcdc_free.vhd** using the **tclkh** and **tclkl** generic values when instantiating **apbslave_tb.vhd**. DCDCCLK frequency is defined in **tb_Dcdc_free.vhd**.

6.1.1.1 Unit Under Test

The Unit Under Test may be one of the following

- Dcdc.vhd - VHDL RTL
- Dcdc_net.vhd - VHDL netlist

One of the above versions may be referenced for the Model Technology Inc. ModelSim simulator, via the library mapping file “libmap” in the **BusTalk/vhdl/modeltech** directory.

6.1.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers, the clock generator and virtual registers. The reader block reads the test vectors from the infile.bif file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the BnRES signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The busline_drivers drives the data bus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK. The virtual registers stimulate the non-AMBA inputs of the Dcdc block and capture values from the non-AMBA outputs for the free-running test. The virtual registers are only used by the free-running clock testbench and they are not used by the production testbench.

6.1.1.3 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timing.vhd file in the **BusTalk/vhdl/tbench** directory.

6.1.1.4 Vector Sets

The DC-DC Converter Interface has two sets of vectors for validating functionality

Free Running Tests

The **Dcdc_free.c** file in the **dcdc_pl160/BusTalk/bustests** directory exercises the functionality of the DC-DC Converter Interface with all the clocks free running. This test uses the Virtual Registers to verify the correct operation of the device. This file is used to generate a BusTalk Input File (*.bif), which is read by the testbench to run the tests.

Production Tests

The **Dcdc_prod.c** file in the **dcdc_pl160/BusTalk/bustests** directory exercises the functionality of the DC-DC Converter Interface in the clock-enabled mode. Please refer to the ARM PrimeCell DC-DC Converter Interface Technical Reference Manual for details on the clock enabling methodology. This test does not use the virtual registers. This file can be used to create production test vectors. This file is used to generate a BusTalk Input File (*.bif), which is read by the testbench to run the tests.

6.1.1.5 Virtual Registers

In the DC-DC Converter Interface, Virtual Registers are used to simulate a system in which the device is being tested. They apply the stimulus to the NON-AMBA inputs as well as capture data on the NON-AMBA outputs. The test bench reads the Virtual Registers for the expected data and indicates any errors found. Refer to the AMBA Compliance Test Suite [Ref. 2] for further details on the Virtual Registers. All the registers are 32 bits wide even though not all 32 bits are used. The following is a table of the virtual registers used along with the list of inputs and outputs each register is used for. The corresponding bit number of each input or output is also listed in the table.

REGISTER	BIT POSITION	INPUTS	OUTPUTS
DCDCDRIVEIN_reg	VR0[0]	DCDCDRIVE0IN	
	VR0[1]	DCDCDRIVE1IN	
DCDCFB_reg	VR1[0]	DCDCFB0	
	VR1[1]	DCDCFB1	
DCDCDRSEL_reg	VR2[0]	DCDCDR0SEL	
	VR2[1]	DCDCDR1SEL	
DCDCDRIVEOUT_reg	VR3[0]		DCDCDRIVE0OUT
	VR3[1]		DCDCDRIVE1OUT
	VR3[2]		DCDCDRIVEOE

Table 6.1.1 Register map of Virtual Registers

6.1.1.6 Generics

The DC-DC Converter Interface testbench provides some configurability options via generics.

Verbosity

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every read command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This generic is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

6.1.1.7 Running Simulations

The **README** file in the **BusTalk/vhdl/tbench** directory gives step by step instructions for running simulations using the clock-enabled or the free-running tests on the VHDL source code.

Toggle coverage numbers when the production test vectors are run on the VHDL in both RTL and netlist forms are available in the above mentioned directory:

Dcdc.prod.bif.rtl.vhdl.vsim.untog

Dcdc.prod.bif.net.vhdl.max.vsim.untog

Dcdc.prod.bif.net.vhdl.min.vsim.untog

A sample .untog file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	= C		
Toggled Node Count	= T		
Untoggled Node Count	C-T = U		
Toggle Coverage	C/T = P %		

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the vhdl RTL or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the **BusTalk/vhdl/tbench/logs** directory. The log files and the toggle reports above were created using the Model Technology Inc ModelSim simulator.

Dcdc.prod.bif.rtl.vhdl.vsim.log

Dcdc.prod.bif.net.vhdl.max.vsim.log

Dcdc.prod.bif.net.vhdl.min.vsim.log

Dcdc.free.bif.rtl.vhdl.vsim.log

Dcdc.free.bif.net.vhdl.max.vsim.log

Dcdc.free.bif.net.vhdl.min.vsim.log

6.1.2 Verilog Bus Compliance testbenches

The ARM PrimeCell DC-DC Converter Interface Verilog test world uses two variants of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbenches are designed to be used with most common Verilog simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbenches are “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted.

Production Testbench

Figure 6.1-5 illustrates the Verilog testbench for running production test vectors.

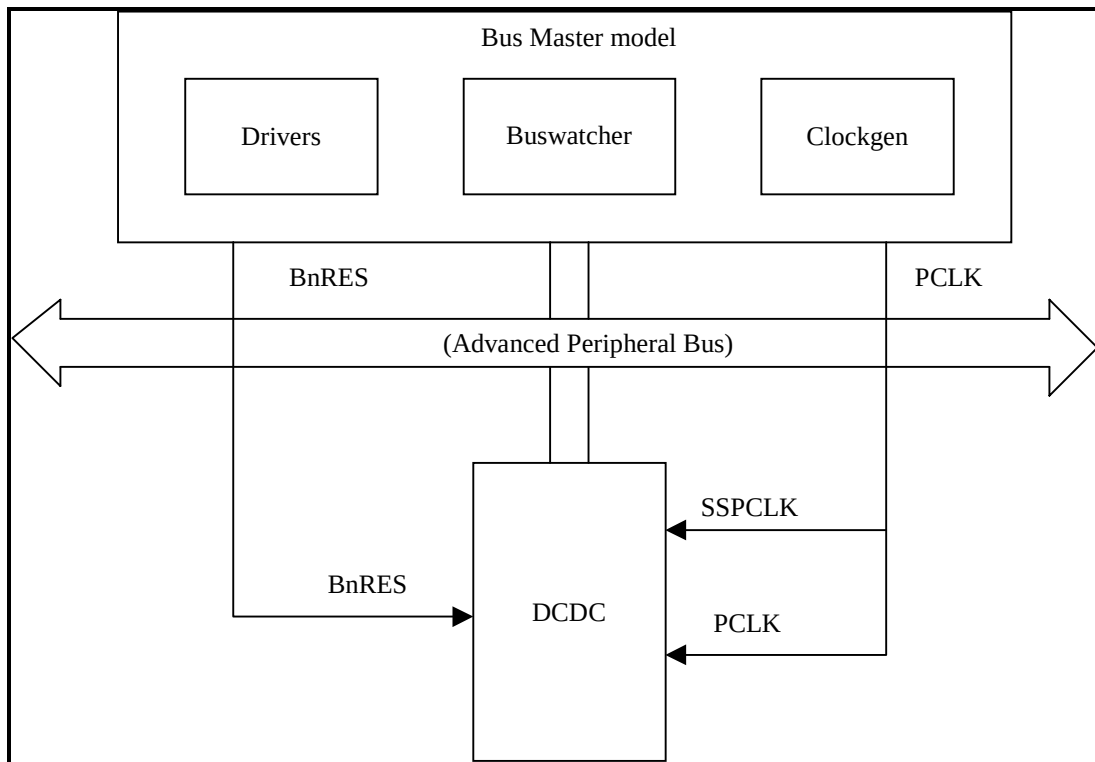


Figure 6.1-5 Verilog testbench for running production test vectors

Figure 6.1-6 illustrates the design hierarchy for the production testbench.

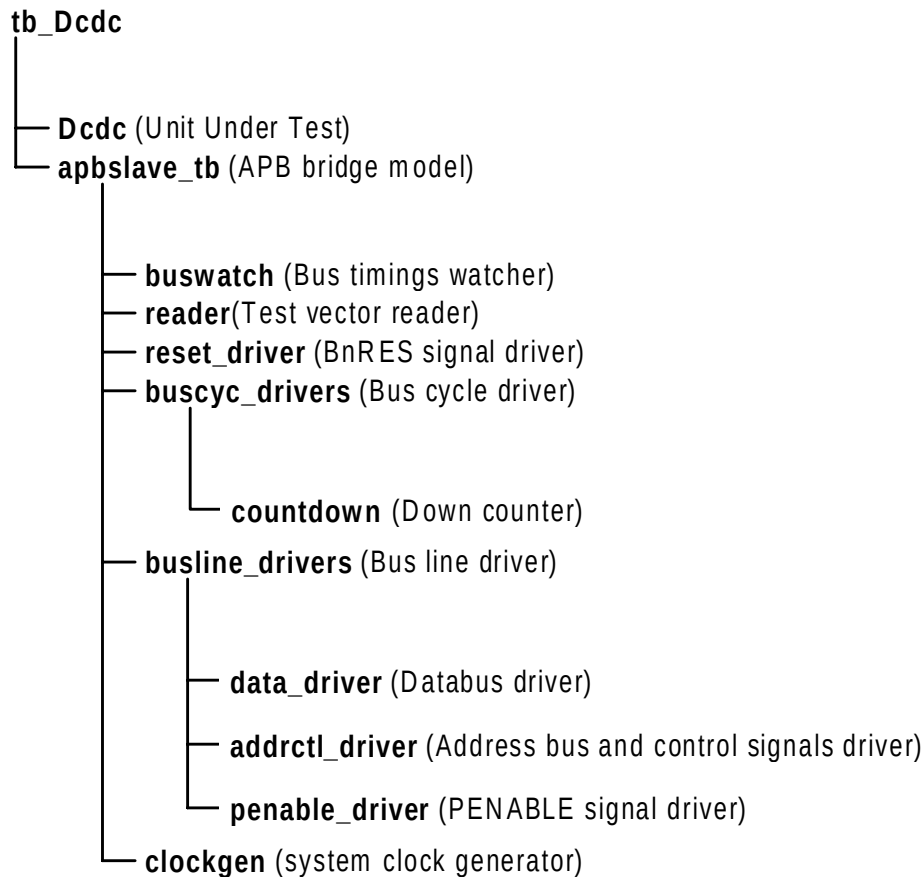


Figure 6.1-6 Testbench hierarchy for production vectors

The testbench, **tb_Dcdc.v** may be used for production level testing using internal clock enabling mechanisms. This block instantiates the DC-DC Converter Interface (Unit Under Test), Virtual Registers block and the behavioural model for the APB bridge. The key point to remember about this testbench is that both PCLK and DCDCLK are fed from the system Bus clock. Since production test vectors use clock enabling to control internal device activity, these two clocks need to run at the same frequency to keep test code frequency independent.

The SCANMODE pin is tied low to keep it from interfering with the test process.

The default test frequency is set at 50MHz. This is exactly the same frequency to which the DCDC was constrained during synthesis. PCLK and DCDCLK are set to the BCLK frequency defined in timing.v using the Tclkh and Tckl 'define values when instantiating apbslave_tb.vhd in tb_Dcdc.v. Note, that the 'define values override the local parameters defined in clockgen.v in the common/ directory.

Functional Verification Testbench

Figure 6.1-7 illustrates the Verilog testbench for tests using free-running clocks.

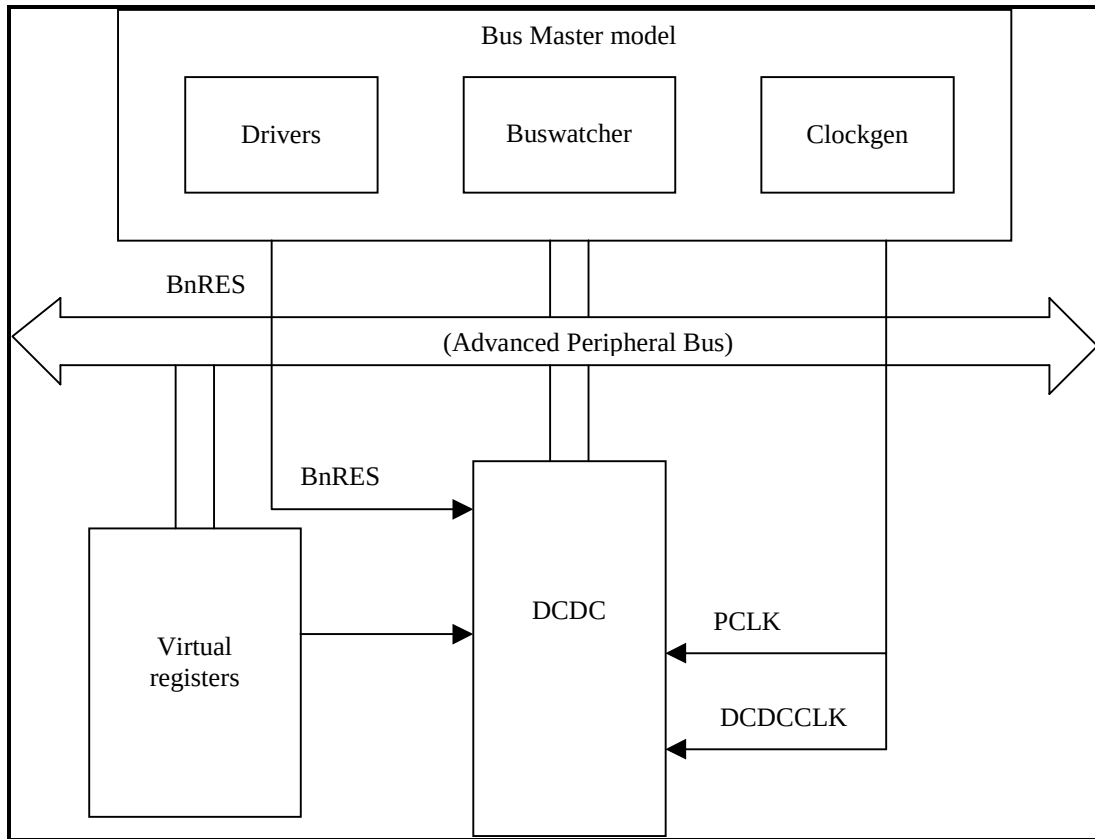


Figure 6.1-7 Verilog testbench for running test vectors using free-running clocks

Figure 6.1-8 illustrates the hierarchy for free-running test vectors.

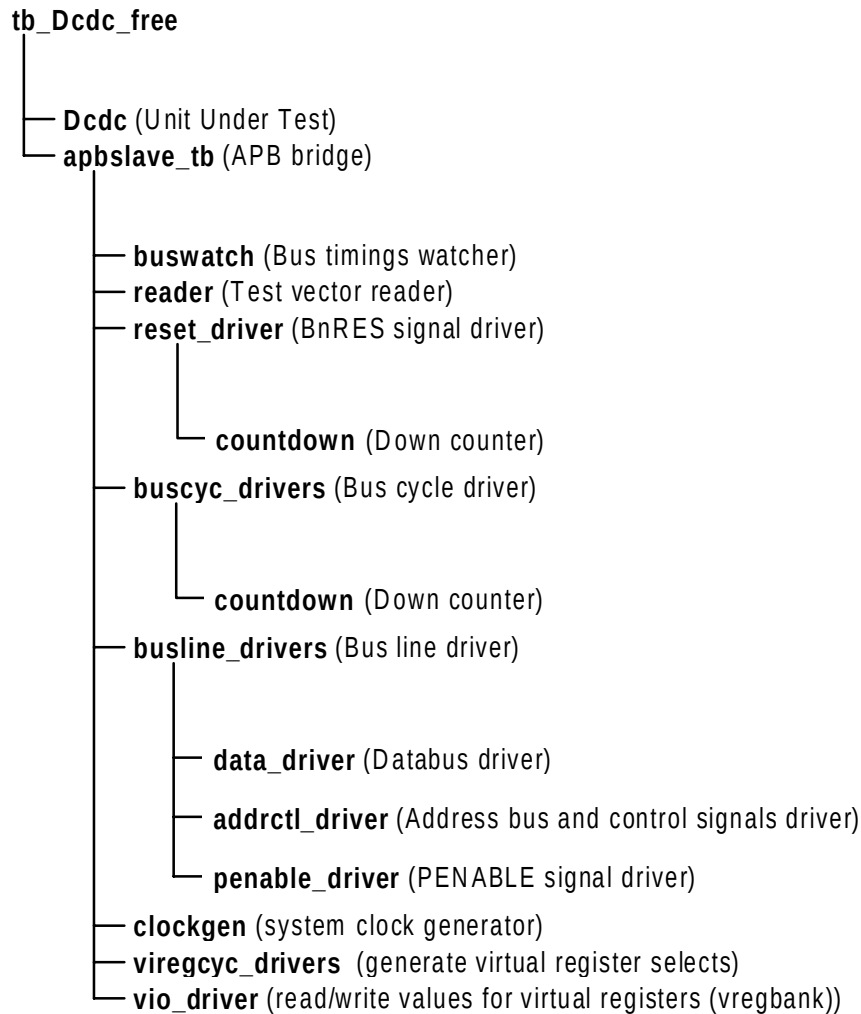


Figure 6.1-8 Testbench hierarchy for free-running vectors

The testbench, **tb_Dcdc_free.v** may be used for functional simulations of the DCDC. This module instantiates the DC-DC Converter Interface (Unit Under Test), Virtual Registers block and the behavioural model for the APB bridge. Unlike the case of **tb_Dcdc.v**, PCLK and DCDCCLK need not be the same frequency.

All non-AMBA inputs to the DCDC block are driven by the virtual registers.

The SCANMODE pin is tied low to keep it from interfering with the test process.

The PCLK frequency has been set to 50 MHz and DCDCCLK was set to 25 MHz. The free-running test program requires a fixed 2:1 clock frequency ratio for PCLK to DCDCCLK. PCLK is set to the BCLK frequency defined in timing.v using the Tclkh and Tckl 'define values when instantiating apbslave_tb.vhd in tb_Dcdc_free.v. Note, that the 'define values override the local parameters defined in clockgen.v in the common/ directory. DCDCCLK frequency is defined in tb_Dcdc_free.vhd.

6.1.2.1 Unit Under Test

The Unit Under Test may be one of the following

- Dcdc.v - Verilog RTL
- Dcdc_net.v - Verilog netlist

One of the above versions may be referenced for the Model Technology Inc. ModelSim simulator, via the library mapping file "libmap" in the **BusTalk/vlog/modeltech** directory.

6.1.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers, the clock generator and virtual registers. The reader block reads the test vectors from the infile.sim file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the BnRES signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e PSW, PSR, PI, PO, etc. The busline_drivers drives the data bus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK. The virtual registers stimulate the non-AMBA inputs of the Dcdc block and capture values from the non-AMBA outputs for the free-running test. The virtual registers are only used by the free-running clock testbench and they are not used by the production testbench.

6.1.2.3 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timing.v file in the **BusTalk/vlog/tbench** directory.

6.1.2.4 Vector Sets

The DC-DC Converter Interface has two sets of vectors for validating functionality

Free Running Tests

The **Dcdc_free.c** file in the **dc_dc_pl160/BusTalk/bustests** directory exercises the functionality of the DC-DC Converter Interface with all the clocks free running. This test uses the Virtual Registers to verify the correct operation of the device. This file is used to generate a BusTalk Input File (but the Verilog version is named *.sim as opposed to the VHDL *.bif version) which is read by the testbench to run the tests.

Production Tests

The **Dcdc_prod.c** file in the **dc_dc_pl160/BusTalk/bustests** directory exercises the functionality of the DC-DC Converter Interface in the clock-enabled mode. Please refer to the ARM PrimeCell DC-DC Converter Interface Technical Reference Manual for details on the clock enabling methodology. This test does not use the virtual registers. This file can be used to create production test vectors. This file is used to generate a BusTalk Input File (but the Verilog version is named *.sim as opposed to the VHDL *.bif version) which is read by the testbench to run the tests.

6.1.2.5 Virtual Registers

In the DC-DC Converter Interface, Virtual Registers are used to simulate a system in which the device is being tested. They apply the stimulus to the NON-AMBA inputs as well as capture data on the NON-AMBA outputs. The test bench reads the Virtual Registers for the expected data and indicates any errors found. Refer to the AMBA Compliance Test Suite [Ref. 2] for further details on the Virtual Registers. All the registers are 32 bits wide even though not all 32 bits are used. The following is a table of the virtual registers used along with the list of inputs and outputs each register is used for. The corresponding bit number of each input or output is also listed in the table.

REGISTER	BIT POSITION	INPUTS	OUTPUTS
DCDCDRIVEIN_reg	VR0[0]	DCDCDRIVE0IN	
	VR0[1]	DCDCDRIVE1IN	
DCDCFB_reg	VR1[0]	DCDCFB0	
	VR1[1]	DCDCFB1	
DCDCDRSEL_reg	VR2[0]	DCDCDR0SEL	
	VR2[1]	DCDCDR1SEL	
DCDCDRIVEOUT_reg	VR3[0]		DCDCDRIVE0OUT
	VR3[1]		DCDCDRIVE1OUT
	VR3[2]		DCDCDRIVEOE

Table 6.1.1 Register map of Virtual Registers

6.1.2.6 Parameters

The DC-DC Converter Interface testbench provides some configurability options via parameters and 'defines.

Verbosity

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every read command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This parameter is used to stop the simulation if a read is unsuccessful. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

6.1.2.7 Running Simulations

The **README** file in the **BusTalk/vlog/tbench** directory gives step by step instructions for running simulations using the clock-enabled or the free-running tests on the Verilog source code.

Toggle coverage numbers when the production test vectors are run on the Verilog in both RTL and netlist forms are available in the above mentioned directory:

Dcdc.prod.bif.rtl.verilog.vsim.untog

Dcdc.prod.bif.net.verilog.max.vsim.untog

Dcdc.prod.bif.net.verilog.min.vsim.untog

A sample .untog file is shown below:

Toggle Report	Node	-> 0	->1
	Signal 1	x	y
	Signal 2	a	b
Total Node Count	=	C	
Toggled Node Count	=	T	
Untoggled Node Count	C-T =	U	
Toggle Coverage	C/T =	P %	

The column "Node" in the .untog file lists the individual nodes that are not toggled when the test vectors are run on the Verilog RTL or netlist forms, whichever the case may be. The column "->0" indicates the number of times that particular node has toggled to 0 and the column "->1" indicates the number of times it has toggled to 1. In the above example, the Node "Signal 1" toggles to 0 "x" times and it toggles to 1 "y" times. The "Total Node Count", indicates the total number of nodes in the block and the "Toggled Node Count" indicates the number of these nodes that are toggled when the test vectors are run. The number "Untoggled Node Count" indicates the number of nodes in the block that are not toggled when the test vectors are run and the "Toggle Coverage" gives the percentage of the total number of nodes in the block that are toggled when the test vectors are run.

Run time logs for all the combinations are available in the following files in the **BusTalk/vlog/tbench/logs** directory. The log files and the toggle reports above were created using the Model Technology Inc ModelSim simulator.

Dcdc.prod.bif.rtl.vlog.vsim.log

Dcdc.prod.bif.net.vlog.max.vsim.log

Dcdc.prod.bif.net.vlog.min.vsim.log

Dcdc.free.bif.rtl.vlog.vsim.log

Dcdc.free.bif.net.vlog.max.vsim.log

Dcdc.free.bif.net.vlog.min.vsim.log

6.1.3 Unification of BusTalk and TicTalk

The DC-DC Converter Interface testbench can automatically produce **.tif** formatted vectors from BusTalk source in addition to the regular **.bif** formatted vectors.

The facility to generate vectors of either format is provided by the addition of **ifdef** control to the **busmacros.c** file in the **BusTalk/bustests** directory of the world. If the **TIF #define** is left uncommented in the **config.h** file in the **bustests** directory, **busmacros.c** will produce **.tif** formatted vectors during compile time. If the **#define** is left commented (default), **.bif** formatted vectors are generated. The following sections detail how each APB Slave Compliance command is interpreted if the **TIF #define** is set.

TestStart()

By default, the **TestStart()** command inserts a pre-determined number of Idle cycles on the APB bus. However, in the Tic world, the startup delays are determined by the testbench itself and not by user written commands. In fact, Tic commands take effect only after BnRES is deactivated. Hence, the busmacros simply prints a comment in the **.tif** file, on encountering a **TestStart()** command in the BusTalk source.

TestEnd()

By default, the **TestEnd()** command appears as the string "TE" in the **.bif** file.

If the TIF #define is set, the **TestEnd()** command is replaced by an **A-A-E** sequence.

PNW()

The **PNW()** command does not have a direct equivalent in the TicTalk world since any access to an APB Slave device address space will cause the APB Bridge to assert the corresponding PSEL along with PSTB. A comment is printed into the **.tif** file for each **PNW()** command.

PSW()

The **PSW()** command appears as an **A-A-W** sequence in the **.tif** file.

PNR()

Like the **PNW()** command, the **PNR()** command does not have a direct equivalent in the TicTalk world. A comment is printed into the **.tif** file for each occurrence of the **PNR()** command.

PSR()

The **PSR()** command appears as an **A-A-R-A** sequence in the **.tif** file. Note that the read is followed by an Address-Only cycle for bus-turnaround.

PI()

The **PI()** command is implemented using an **A-A-L** sequence in the **.tif** file.

RES()

Since the assertion of BnRES is not under command control in the TicTalk world, the **RES()** command simply prints out a comment in the **.tif** file. BnRES assertion occurs automatically under testbench control at the start of the simulation.

PO()

Note that the POLL command does not have any equivalent in the TicTalk world. Hence, BusTalk source code to be converted into **.tif** formatted vectors should not include any **PO()** commands.

6.1.4 Vector Generation

A UNIX make file (***make sourcefilename*** without the .c extension) in the ***bustests*** directory will create BusTalk Input Files, *.***bif*** (for VHDL) and *.***sim*** (for verilog) vectors in the ***bustests/invec*** directory. This is the default setting.

To create TicTalk Input Files, *.***tif*** (for VHDL) vectors and *.***sim*** (for Verilog) from the same source, the following two changes need to be made.

the ***#define TIF*** needs to be set in the ***config.h*** file in the ***bustests*** directory.

setenv VECT_TYPE TIF at the UNIX command line.

Now ***make sourcefilename*** (without the .c extension) will create ***.tif*** formatted vectors in the ***bustests/invec*** directory.

To revert back to ***.bif*** generation, do the following

/* #define TIF */ in config.h

unsetenv VECT_TYPE

6.1.5 AMBA TICTalk testbenches

In addition to the BusTalk testbenches, TICTalk VHDL and Verilog testbenches are supplied to apply TICTalk production test vectors to the DCDC. The TICTalk testbenches is based on a subset of components from the ARM Example AMBA System (EASY). For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

The TICTalk production test vectors used with these testbenches can be applied to the DCDC when it part of an AMBA system design.

Figure 6.1-9 illustrates the testbench to apply TICTalk test vectors.

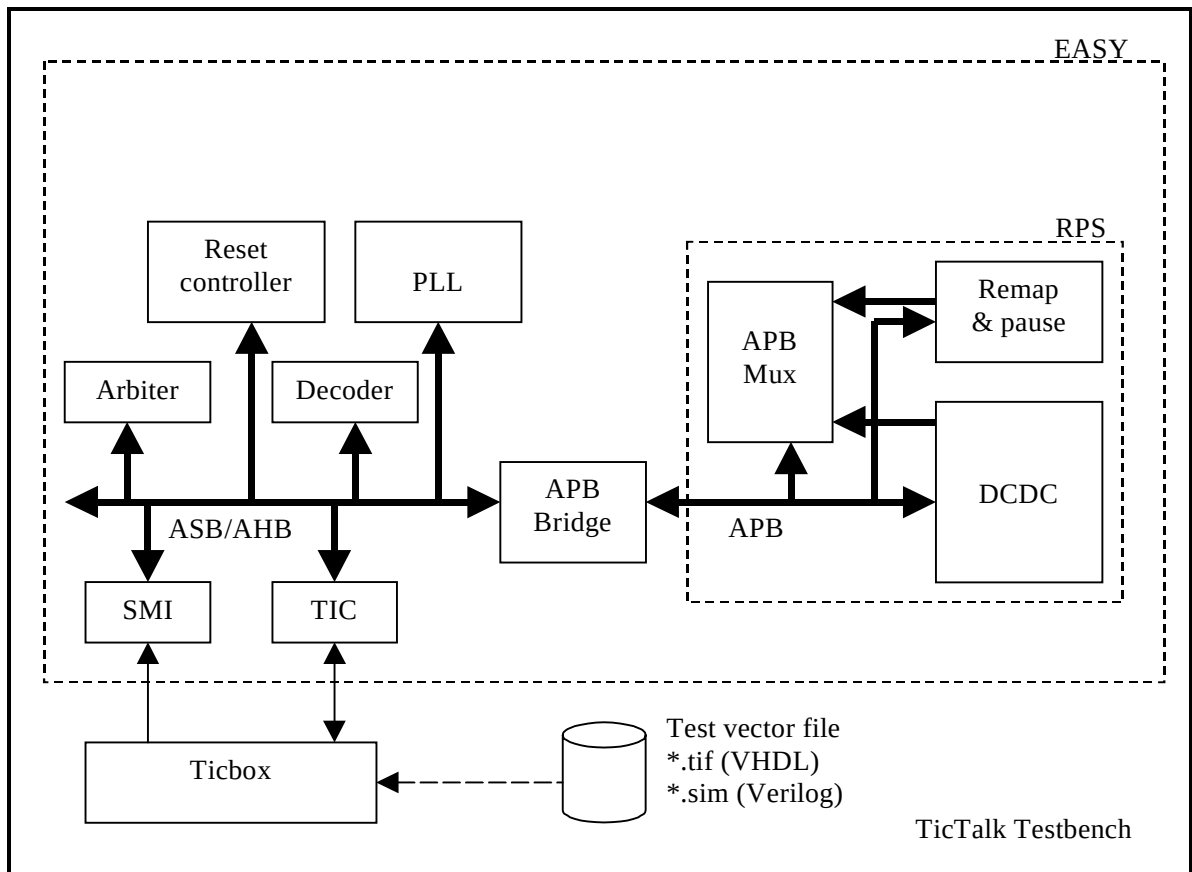


Figure 6.1-9 Testbench for running TICTalk production test vectors

Figure 6.1-10 illustrates the hierarchy for the TicTalk testbench.

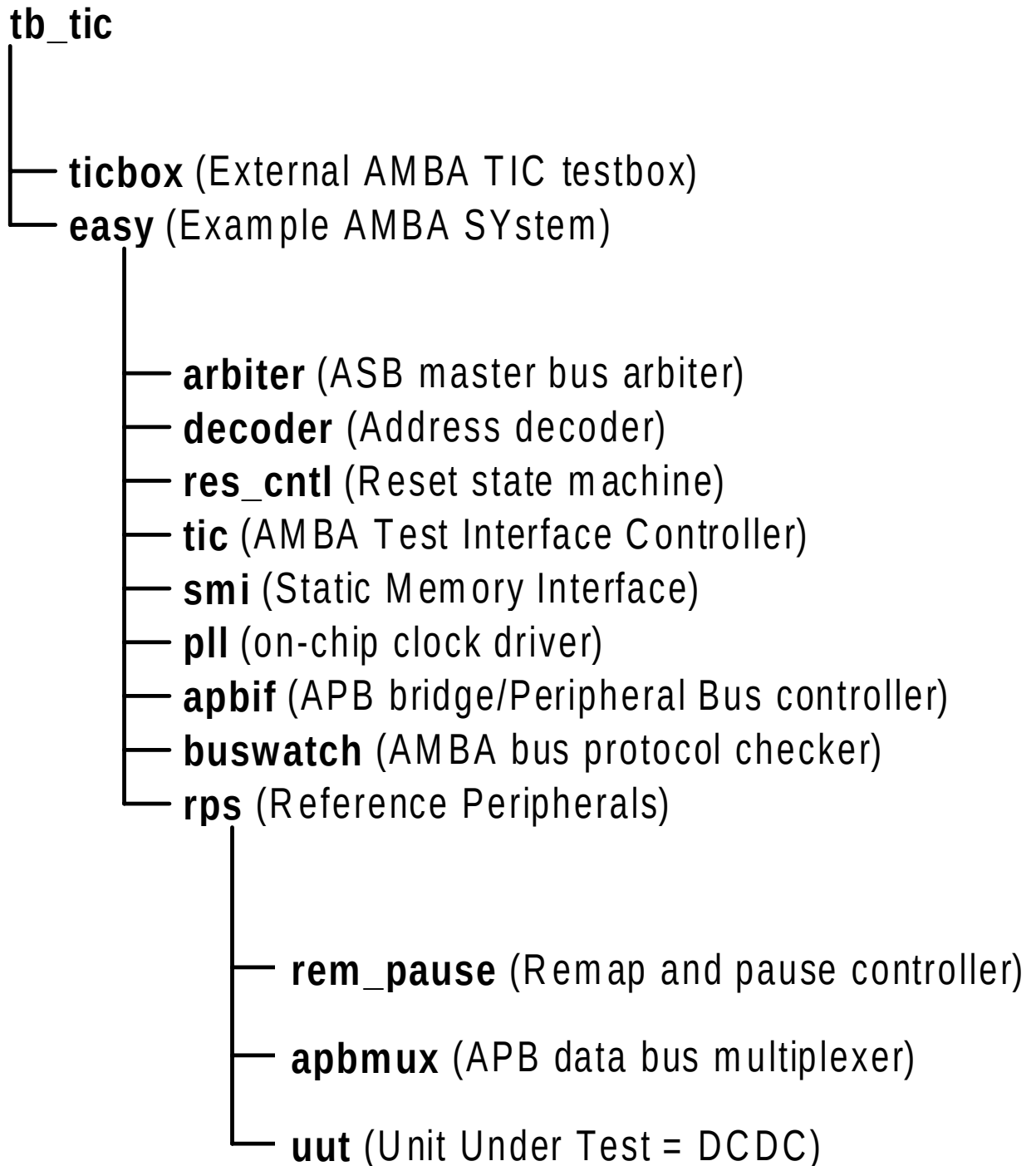


Figure 6.1-10 Testbench hierarchy for-running TicTalk test vectors

6.2 Free Running Functional Verification Tests

The following sections describe all the test cases that were run during the validation of the DC-DC Converter Interface. These tests use the DC-DC Converter Interface Virtual Registers. As part of this test 15 sub tests are run. An explanation of each test follows.

6.2.1 Register Reset Tests

Reset Value Tests

After a reset, each of the following registers are read to check if they contain the specified reset value.

Register Name	Reset Value
PMPCON0	0x00
PMPCON1	0x00
PMPFREQ	0x00
PMPTCER	0x00
PMPTCR	0x00
PMPTISR	0x00
PMPTOCR	0x04

Read/Writeability of the writeable registers.

A data pattern is written to each of the writeable registers PMPCON0, PMPCON1, PMPFREQ, PMPTCER, PMPTCR, and PMPTISR.

The register contents are read back to see if it contains the written value. If the values are the same then the register is declared to be writeable and readable and the test passes.

6.2.2 Positive Polarity Drive 0 1800 KHz Functional Test with DRV0Sel = 1

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Positive Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at specific times in the test for the expected values.

6.2.3 Positive Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 0

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Positive Polarity and **DRV0SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.2.4 Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Positive Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at specific times in the test for the expected values.

6.2.5 Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 0

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Positive Polarity and **DRV1SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.2.6 Negative Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 1

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Negative Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.2.7 Negative Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Negative Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.2.8 FEEDBACK0 CHECK

In this test the Virtual Register is programmed so that the **DCDCFB0** is negated. This should ensure that **DCDCDRIVE0OUT** remains Low at all appropriate times while running the Drive 0 1800 KHz Functional test. If the **DCDCDRIVE0OUT** output remains low, the test is deemed to have passed.

6.2.9 FEEDBACK1 CHECK

In this test the Virtual Register is programmed so that the **DCDCFB1** is negated. This should ensure that **DCDCDRIVE1OUT** remains Low at all appropriate times while running the Drive 1 1800 KHz Functional test. If the **DCDCDRIVE1OUT** output remains low, the test is deemed to have passed.

6.2.10 Drive 0 900 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 900 KHz with Positive Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.2.11 Drive 1 900 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 900 KHz with Positive Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.2.12 Drive 0 225 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 225 KHz with Positive Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.2.13 Drive 1 225 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 225 KHz with Positive Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.2.14 Drive 0 95 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 94.7 KHz with Positive Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.2.15 Drive 1 95 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 94.7 KHz with Positive Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.3 Clock-enabled Production Tests

All the sub tests within the Production test verify the correct operation of the DC-DC Converter Interface in the Clock enabled mode. These tests can be used to produce the production vectors. All the tests are run with DC-DC Converter Interface in the Clock Enabled mode. Please refer to the DC-DC Converter Interface for further information on Clock Enabling. There are 16 Sub Tests as part of this test.

6.3.1 Register Reset Tests

Reset Value Tests

After a reset, each of the following registers are read to check if they contain the specified reset value.

Register Name	Reset Value
PMPCON0	0x00
PMPCON1	0x00
PMPFREQ	0x00
PMPTCER	0x00
PMPTCR	0x00
PMPTISR	0x00
PMPTOCR	0x04
PMPFC0	0x00
PMPFC1	0x00
PMPDRVCNT	0x00

Read/Writeability of the writeable registers.

A data pattern is written to each of the writeable registers PMPCON0, PMPCON1, PMPFREQ, PMPTCER, PMPTCR, and PMPTISR.

The register contents are read back to see if it contains the written value. If the values are the same then the register is declared to be writeable and readable and the test passes.

6.3.2 Counter Operation Test

As a part of this sub test, the device is configured such that the Frequency Counters are placed in the nibble increment mode and the test verifies the correct operation of these counters. The counters are placed in the nibble increment mode by programming the PMPTMR register. Please refer to the ARM PrimeCell DC-DC Converter Interface Technical Reference Manual for details on programming the DC-DC Converter Interface.

6.3.3 Positive Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 1

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Positive Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at specific times in the test for the expected values.

6.3.4 Positive Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 0

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Positive Polarity and **DRV0SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.3.5 Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Positive Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at specific times in the test for the expected values.

6.3.6 Positive Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 0

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Positive Polarity and **DRV1SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.3.7 Negative Polarity Drive 0 1800 KHz Functional Test with DRV0SEL = 1

This test configures the **DCDCDRIVE0OUT** output to run at 1800 KHz with Negative Polarity and **DRV0SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.3.8 Negative Polarity Drive 1 1800 KHz Functional Test with DRV1SEL = 1

This test configures the **DCDCDRIVE1OUT** output to run at 1800 KHz with Negative Polarity and **DRV1SEL** held High. This ensures that the duty cycle is read from the Upper 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.3.9 FEEDBACK0 Check

In this test the **PMPTISR** Register is programmed so that the **DCDCFB0** is negated. This should ensure that **DCDCDRIVE0OUT** remains Low at all appropriate times while running the Drive 0 1800 KHz Functional test. If the **DCDCDRIVE0OUT** output remains low, the test is deemed to have passed.

6.3.10 FEEDBACK1 Check

In this test the **PMPTISR** register is programmed so that the **DCDCFB1** is negated. This should ensure that **DCDCDRIVE1OUT** remains Low at all appropriate times while running the Drive 1 1800 KHz Functional test. If the **DCDCDRIVE1OUT** output remains low, the test is deemed to have passed.

6.3.11 Drive 0 900 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 900 KHz with Positive Polarity and **DRV0SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.3.12 Drive 1 900 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 900 KHz with Positive Polarity and **DRV1SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.3.13 Drive 0 225 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 225 KHz with Positive Polarity and **DRV0SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.3.14 Drive 1 225 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 225 KHz with Positive Polarity and **DRV1SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.

6.3.15 Drive 0 95 KHz Functional Test

This test configures the **DCDCDRIVE0OUT** output to run at 94.7 KHz with Positive Polarity and **DRV0SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON0** Register. The **DCDCDRIVE0OUT** is checked at given times in the test for the expected values.

6.3.16 Drive 1 95 KHz Functional Test

This test configures the **DCDCDRIVE1OUT** output to run at 94.7 KHz with Positive Polarity and **DRV1SEL** held Low. This ensures that the duty cycle is read from the Lower 4 bits of **PMPCON1** Register. The **DCDCDRIVE1OUT** is checked at given times in the test for the expected values.